The Object RISC Architecture

Architecture Reference, Revision 0.1

The Object RISC Architecture Group

1986

Contents

Volume I — Overview

1. Introduction

Object RISC is a 32-bit load-store reduced instruction set architecture designed for tightly-coupled multiprocessor systems. It draws on the work emerging from Stanford and Berkeley in the early 1980s — small instruction sets, fixed instruction widths, single-cycle execution, and pipelined implementations — and combines those principles with first-class hardware support for objects and a crossbar-based processor interconnect.

The architecture has three design goals, which it treats as equally important:

1. A single Object RISC processor should be cheap, simple, and fast to build with the process technologies available in the late 1980s.
2. A collection of Object RISC processors connected by a crossbar should present a coherent, location-transparent object space to software, without requiring a separate communications co-processor or a software message-passing layer in the hot path.
3. The combination of hardware and the runtime services common to any operating system should present a single, stable upward interface, so that software written for one Object RISC configuration runs unmodified on others. The layer that provides this interface is part of the architecture, and is described in Section 6 as *System Firmware*.

We believe these goals are compatible. The first disciplines the second: every feature added to support multiprocessing must justify itself against the cost it imposes on the single-processor pipeline. The third disciplines both: features that cannot be made cheap and that vary between implementations are pushed out of silicon and into firmware, where they can be specified once and reimplemented appropriately for each generation of hardware.

The result is a design that looks, on a per-processor basis, very much like its RISC contemporaries — but whose register file, instruction set, trap model, and surrounding firmware layer are quietly shaped by the assumption that the processor is one of many.

2. Influences and Lessons Learned

Object RISC is not the first architecture to attempt hardware support for objects. Intel's iAPX 432, introduced in 1981, was the most ambitious prior attempt, and its commercial failure casts a long shadow over this work. The 432 demonstrated that an object model implemented in microcode, with variable-length instructions and an interpretive execution model, could not compete on price

or performance with conventional designs of the same era — let alone with the new RISC processors then emerging.

We draw three lessons from the 432:

- **The object model belongs in the register file, not in microcode.** Object references should be values that the programmer-visible machine can hold, compare, and pass around. The cost of dereferencing an object reference must be a small, bounded number of pipeline cycles, not a microcoded sequence.
- **Bounds and capability checks must be free in the common case.** If accessing a field of an object costs more than accessing a word of memory through a base register, programmers will route around the object system, and the hardware investment is wasted.
- **Generality is the enemy of throughput.** The 432 supported an elaborate type system in hardware. Object RISC supports the minimum primitive — a bounded, capability-tagged region of memory — and leaves type structure to the compiler and runtime.

The crossbar-based multiprocessor model is influenced by the Carnegie Mellon C.mmp work of the previous decade, and more recently by the INMOS Transputer, whose serial links between processors demonstrated that inter-processor communication could be a primitive of the architecture rather than a peripheral. Object RISC differs from the Transputer in two respects: communication is by addressed message rather than by named channel, and the interconnect is a crossbar rather than a fixed graph of point-to-point links.

The two-level naming arrangement — a global object namespace above, per-task virtual address spaces below — is influenced by Apollo Computer's Domain system, in which long-form object identifiers name storage uniformly across a network of workstations and processes map those objects into their own thirty-two-bit virtual address spaces on demand. Object RISC compresses Apollo's network into a crossbar and the host operating system's mapping machinery into firmware, but the underlying proposition is the same: the namespace in which storage is named must be larger than the address space in which any single task computes, and the bridge between the two is built at runtime rather than designed into the address layout. A 32-bit architecture intended to scale beyond a single processor cannot, in our view, do otherwise.

3. The Processor

A single Object RISC processor is conventional in most respects.

- **Word size.** 32 bits. Addresses, integers, and instructions are all 32 bits wide. Byte and halfword loads and stores are supported but internally widened.
- **Endianness.** Big-endian. Byte 0 of a word is the most significant.
- **Register file.** Thirty-two general-purpose registers, `R0` through `R31`. `R0` reads as zero and discards writes, in the manner of MIPS. In addition, sixteen *object registers*, `O0` through `O15`, hold object references. Object registers are a distinct register class with their own load, store, and move instructions; they cannot be used as general-purpose integer registers, and integer registers cannot be dereferenced as objects. This separation is the central architectural decision of Object RISC and is discussed further in Section 4.

- **Instruction format.** All instructions are 32 bits, aligned on 4-byte boundaries. Three primary formats are defined: register-register (R-type), register-immediate (I-type), and jump (J-type), in the style of the Stanford MIPS design.
- **Pipeline.** A five-stage pipeline is the reference implementation: instruction fetch, decode and register read, execute, memory access, and write-back. Branches are delayed by one instruction slot. Loads have a one-cycle delay slot in the reference implementation; the compiler is responsible for scheduling around it.
- **Condition codes.** None. Comparisons produce a value in a general register, and conditional branches test a register against zero or against another register. This avoids the pipeline serialization that condition-code architectures suffer at branches and simplifies the trap model.
- **Address translation.** Each task executes in a private 32-bit virtual address space. The processor includes a translation lookaside buffer and a hardware page-table walker; page tables are per-task and are installed by System Firmware on context switch, with page faults trapping to firmware. The translation hardware serves ordinary loads and stores against the current task's address space. Loads and stores through an object register take a separate translation path described in Section 4, and the two paths share no hardware state.
- **Privilege.** Three modes: *user*, *supervisor*, and *firmware*. The object register file is available in all three, but capability rights checked at object dereference distinguish trusted from untrusted code. Firmware mode — the privilege at which System Firmware (Section 6) executes — is the only mode permitted to mint new object references, to install entries in the crossbar routing table, or to address the device space directly. User and supervisor code reach those facilities by trapping into firmware via the `CALL` instruction.

A reference implementation in 1.5-micron CMOS is expected to fit in approximately 110,000 transistors and to run at 16 to 20 MHz, comparable to the first-generation MIPS R2000 and SPARC parts.

4. Objects

An *object*, in Object RISC, is a contiguous region of memory together with a hardware-enforced descriptor. The descriptor records the object's base address, its length in bytes, its type tag (an opaque 16-bit value interpreted only by software), the home processor on which it physically resides, and a set of capability bits — read, write, execute, send, and revoke.

An *object reference* is a 64-bit value held in an object register. It combines an index into the system-wide object table with a generation counter that is incremented when the underlying object is freed. The generation counter prevents a stale reference from accidentally naming a later object that happens to reuse the same table slot. Object references are produced only by System Firmware, in response to primitive calls issued from user or supervisor code; they are otherwise opaque to the caller, which can copy them between object registers and pass them as arguments but cannot manufacture them.

Loads and stores through an object register take an offset and a width. The hardware checks, in parallel with address computation, that the offset and width fall within the object's length and that the requesting processor holds the appropriate capability bit. A failed check raises a synchronous

trap before the memory access is committed. In the common case — a valid access to a local object — these checks add no cycles to the load or store; the descriptor is held in a small associative cache local to the processor, distinct from the per-task TLB and shared across all tasks resident on the processor.

For sustained access patterns a task may instead *map* an object, or a window into one, into its own virtual address space through the appropriate firmware primitive. Firmware installs page-table entries that resolve the chosen virtual range to the object's physical storage on the home processor, honouring the capability bits recorded in the descriptor. Subsequent ordinary loads and stores within that range run at full MMU speed, with no further reference to the object register file. The choice between the two access paths is one of working-set characteristics, not of capability: the protection guarantees are identical in either case. Mapping is restricted to objects whose home is the local processor; remote objects remain accessible through the object register file, which transparently routes the access across the crossbar.

This two-path arrangement is the practical consequence of being a 32-bit architecture in a multiprocessor system. A flat space of four gigabytes cannot accommodate, even in the medium term, the aggregate storage reachable across an Object RISC crossbar. Object references therefore live in a global namespace far larger than any single task can address, and per-task address spaces are populated from that namespace by mapping at runtime. Within a task, ordinary pointer arithmetic remains 32-bit and remains fast; between tasks, and between processors, storage is named by reference rather than by address.

This is what we mean by "Object RISC": objects are first-class hardware entities, but they are kept architecturally lean. There is no inheritance, no method dispatch, no garbage collector, and no type lattice in the hardware. Those mechanisms belong to the compiler and the runtime, where they can evolve without an instruction-set revision.

5. The Crossbar Interconnect

An Object RISC system comprises one or more processors, one or more memory modules, and a *crossbar* connecting them. The reference configuration is a 16-port crossbar with single-cycle arbitration and a peak aggregate bandwidth proportional to the port count and the cycle time. Smaller and larger configurations are permitted; the architecture defines the protocol on the wire, not the physical topology.

The crossbar is visible to software in two ways.

First, every object reference names a *home processor*. When a processor issues a load or store through an object reference whose home is itself, the access is local and proceeds through the cache and memory hierarchy in the usual way. When the home is a different processor, the access is packaged as a message, routed across the crossbar to the home, executed there against the local memory, and the response routed back. The processor stalls the issuing instruction in the same way it would stall on a cache miss. From the programmer's perspective the load or store behaves identically; it is merely slower.

Second, a `SEND` instruction transmits an explicit message — an object reference together with a small payload of general-register values — to the home processor of that reference. The receiving processor delivers the message to a handler that System Firmware has installed on behalf of the addressed object. This primitive is the intended substrate for higher-level constructs: remote procedure calls, actor-style message passing, and operating-system requests across the crossbar all reduce to `SEND`.

The crossbar arbiter, the routing table, and the message buffers are specified by the architecture but are not part of any individual processor. A small system may implement them on a single companion chip; a larger system may distribute them across several. The protocol is the contract.

6. System Firmware

The hardware described above is necessary but not sufficient to constitute an Object RISC system. The architecture also defines a *virtual machine interface*: a fixed set of system primitives, invoked from ordinary code via a single trap-style `CALL` instruction, that together specify the services any conforming implementation must provide. The layer that implements them — and that, in any multiprocessor or multi-tenant configuration, also acts as the hypervisor under which guest operating systems are hosted — is called *System Firmware*.

The primitive set covers the work that is common to every system above the bare metal but that varies in detail between implementations:

- **Task management.** Creation, scheduling, suspension, and termination of tasks; binding of tasks to processors; primitives for synchronization between tasks both on the same processor and across the crossbar.
- **Memory management.** Allocation and deallocation of objects from the physical memory pool; assignment of objects to home processors; migration and revocation of references; mapping and unmapping of objects into the virtual address space of a task, and modification of the access rights with which they are mapped; service of page faults raised by the address translation hardware.
- **Communications.** Installation of handlers for incoming `SEND` messages; delivery, queuing, and flow control of messages whose recipient is not currently scheduled; routing-table maintenance.
- **Device and I/O interaction.** Discovery and naming of devices as objects; mediation of interrupts; mapping of device registers behind capability-checked object references so that drivers, like all other code, address hardware only through the object register file.
- **Time, clocks, and console services**, and the small remainder of facilities that any operating system would otherwise have to reinvent.

System Firmware runs on the same processors as ordinary code, in the firmware privilege mode introduced in Section 3. It is the only software with direct access to the object table, the crossbar routing table, the device address space, and the physical memory map. Supervisor-mode code (an operating system) and user-mode code (an application) reach those resources only through

primitive calls. The primitive interface is part of the architecture; the firmware that implements it is supplied by the hardware vendor or system integrator.

This stratification has three consequences worth stating up front.

The hardware ISA can be smaller. Many features that would conventionally belong to the processor — virtual address translation, paging, scheduling support, interrupt prioritization, communication buffer management — are firmware concerns. The processor is not required to implement them in silicon, only to provide the trap mechanism by which the firmware is reached.

The system has a stable upward interface. Code written to the primitive set is portable across Object RISC implementations and across configurations of different scale, from a single processor with a small crossbar to a sixteen-processor system. The firmware absorbs the differences, and a recompilation against a new hardware generation should not, in general, be necessary.

Multiple operating systems can coexist. Because the firmware is the hypervisor, an Object RISC system can host more than one operating system simultaneously, each in its own protection domain, sharing the processors and crossbar through firmware-mediated scheduling. We expect this capability to be of particular interest in research and educational settings, and in the gradual migration of existing software bases.

The full specification of the primitive interface — the call numbers, parameter conventions, return values, and trap conditions — is the subject of Volume VI of this reference. The present volume notes only that, in the architecture as a whole, "Object RISC system" refers to the combination of conforming hardware and conforming firmware: neither is meaningful alone.

7. What Object RISC Is Not

To narrow the scope of subsequent volumes, we list explicitly what the architecture does not provide.

- **Cache coherence across the crossbar.** Processors cache only local memory. Remote accesses are uncached at the issuing processor and rely on the home processor's cache. A future revision may relax this; the current revision does not.
- **Hardware floating point.** A floating-point co-processor interface is reserved, but the reference processor does not implement floating point. Software emulation traps are provided.
- **Demand paging to backing store.** The architecture defines a page fault trap and routes it to firmware, but specifies no convention by which virtual pages are backed with disk or other slow storage. Whether to support such a mechanism — and the policy by which dirty pages are evicted, prefetched, or pinned — is a firmware concern that may legitimately differ between implementations.
- **Garbage collection.** The architecture provides no hardware support for tracing or reclaiming unreferenced objects. References are released explicitly through firmware primitives, and stale references are caught at use through the generation counter described in Section 4.

- **Dynamic dispatch.** The hardware does not implement method lookup, inheritance, or any other form of indirect call beyond the `JALR` instruction. The object register file makes such mechanisms cheap to build in software; it does not implement them.

8. Roadmap of Subsequent Volumes

This volume gives the high-level overview. Detailed specifications are deferred to:

- **Volume II — Instruction Set.** The full instruction encoding, semantics, and trap conditions for the integer and object subsets, including the `CALL` instruction by which firmware is reached.
- **Volume III — Object System.** The descriptor format, the object table, capability semantics, and the firmware primitives for minting, revoking, and migrating references.
- **Volume IV — Interconnect Protocol.** The wire-level message format, arbitration, routing-table semantics, and timing requirements for the crossbar.
- **Volume V — Reference Implementation.** A pipeline-level description of a representative single-issue processor and a 16-port crossbar, including expected gate counts and cycle times.
- **Volume VI — System Firmware Interface.** The complete specification of the system primitive set: call numbers, parameter conventions, return semantics, trap conditions, and the requirements that a conforming firmware implementation must meet on top of any conforming hardware.

Volume II — Instruction Set

1. Scope

This volume specifies the instructions executed by an Object RISC processor in user, supervisor, and firmware modes. It covers the integer subset, the operations on object registers, the inter-processor communication primitive `SEND`, the firmware-call primitive `CALL`, the small number of privileged instructions, and the trap and exception behaviour common to all of them.

It does not cover the wire-level format of crossbar messages (Volume IV), the cycle-by-cycle behaviour of any particular implementation (Volume V), or the firmware primitive set reached through `CALL` (Volume VI). Where those volumes describe details visible to the programmer, this volume states the architectural contract; conforming implementations may differ in timing and microarchitecture but must agree on the contract.

The reader is assumed to be familiar with Volume I.

2. Notation

In the descriptions that follow:

- `Rs`, `Rt`, `Rd` denote general-purpose registers, with `Rs` and `Rt` reading their values and `Rd` receiving the result.
- `Os`, `Ot`, `Od` denote object registers under the same convention.
- `imm` denotes a 16-bit immediate field, sign-extended to 32 bits unless noted otherwise; `uimm` is the same field zero-extended.
- `target` denotes the 26-bit target field of a J-type instruction.
- `offset` denotes a 16-bit signed displacement in bytes.
- ← denotes assignment; `||` denotes bitwise concatenation; `M[a]` and `M[a:b]` denote memory contents at byte address `a`, the latter for a range.
- All addresses are byte addresses. Multi-byte accesses must be naturally aligned; misalignment raises a synchronous trap.
- Instruction encodings are written most-significant bit first, in the big-endian byte order used throughout the architecture.

3. Register Model and Calling Conventions

3.1 General-Purpose Registers

The processor provides thirty-two 32-bit general-purpose registers, `R0` through `R31`. `R0` is hardwired to zero: writes are silently discarded and reads return the constant zero. `R31` is, by convention, the link register written by `JAL` and `JALR`; it has no special hardware treatment outside of those instructions.

The standard calling convention partitions the register file as follows:

Register	Use
`R0`	constant zero
`R1`	reserved as assembler temporary
`R2`–`R3`	integer return values
`R4`–`R7`	integer arguments
`R8`–`R15`	caller-saved temporaries
`R16`–`R23`	callee-saved
`R24`–`R28`	additional caller-saved temporaries
`R29`	stack pointer (`SP`)
`R30`	frame pointer (`FP`)
`R31`	link register (`RA`); set by `JAL`, `JALR`

The convention is recommended, not enforced by hardware. Code that violates it executes correctly but does not interoperate with the standard runtime.

3.2 Object Registers

The processor provides sixteen object registers, `O0` through `O15`, each holding a 64-bit object reference whose internal layout is specified in Volume III. `O0` is hardwired to the *null reference*: writes are silently discarded and reads return the null constant. The null reference compares equal only to itself and traps on any attempt to dereference it.

The standard calling convention partitions the object register file as follows:

Register	Use
`O0`	null reference
`O1`–`O4`	object arguments and first object return value
`O5`–`O8`	caller-saved temporaries
`O9`–`O12`	callee-saved
`O13`–`O15`	additional caller-saved temporaries

The same convention governs the payload of `SEND`: object arguments to the remote handler are placed in `O1`–`O4` and integer arguments in `R4`–`R7`, exactly as for a local call. A remote invocation therefore differs from a local one only in the instruction used to issue it.

3.3 Program Counter and Status

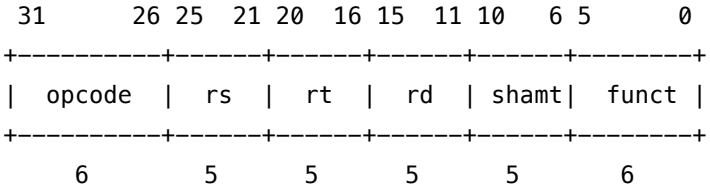
The program counter is not directly readable as a general-purpose register; its value is captured by `JAL`, `JALR`, and the trap mechanism. A small number of architecturally-visible status registers (current mode, interrupt enable mask, exception program counter, exception cause, and the like) are accessed through the privileged `LCTRL` and `SCTRL` instructions described in Section 12. Their detailed contents are specified in Volume V.

4. Instruction Formats

All instructions are 32 bits and aligned on 4-byte boundaries. Misaligned instruction fetch raises a bus-error trap.

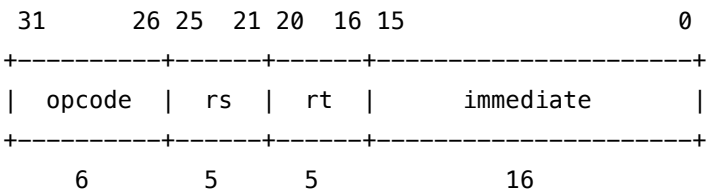
Four formats are defined.

4.1 R-Type (Register-Register)



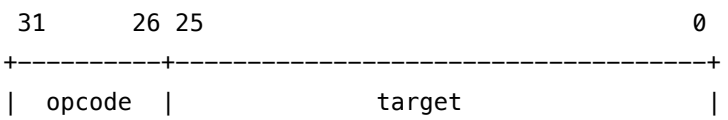
The major opcode for all R-type instructions is `0x00` (`SPECIAL`); the 6-bit `funct` field selects the operation.

4.2 I-Type (Register-Immediate)



Used for arithmetic with immediates, conditional branches, and loads and stores through general-purpose registers.

4.3 J-Type (Jump)



```
+----------+-----------------------------------+
     6                      26
```

The 26-bit `target` is shifted left by two and combined with the upper four bits of the address of the delay-slot instruction to form the absolute branch target.

4.4 O-Type (Object Operations)

Two related formats serve the object register file.

The *object register-register* form is used for operations whose only operands are object registers and (optionally) general registers:

```
 31      26 25 22 21 18 17  13 12   8 7   4 3   0
+----------+----+----+------+------+-----+-----+
|  opcode  | os | ot |  rd  |  rs  |funct| rsv |
+----------+----+----+------+------+-----+-----+
     6       4    4     5      5     4     4
```

The *object load-store* form replaces the trailing fields with a 16-bit signed offset for object-relative memory access:

```
 31      26 25 22 21    16 15                  0
+----------+----+--------+----------------------+
|  opcode  | os |  rt'   |        offset        |
+----------+----+--------+----------------------+
     6       4      6               16
```

The `rt'` field is six bits at positions 21:16, of which the high bit (bit 21) is reserved and must be zero in this revision; the low five bits hold the GPR number, as in every other I-type. Future widening of the register file may consume the reserved bit.

5. Integer Computation

The integer subset is conventional for its era. Three-operand register forms are encoded as R-type with major opcode `SPECIAL` and a per- operation `funct`; immediate forms are encoded as I-type.

Mnemonic	Effect
`ADD Rd, Rs, Rt`	`Rd ← Rs + Rt`; trap on signed overflow
`ADDU Rd, Rs, Rt`	`Rd ← Rs + Rt`; no overflow trap
`SUB`, `SUBU`	as `ADD`/`ADDU`, with subtraction
`AND`, `OR`, `XOR`	bitwise
`NOR Rd, Rs, Rt`	`Rd ← ¬(Rs v Rt)`
`SLL Rd, Rt, sa`	logical shift left by 5-bit `shamt`
`SRL`, `SRA`	logical and arithmetic shift right
`SLLV`, `SRLV`, `SRAV`	variable shift; count is low five bits of `Rs`

Mnemonic	Effect
`SLT Rd, Rs, Rt`	`Rd ← 1` if `Rs < Rt` (signed) else `0`
`SLTU`	as `SLT`, unsigned
`MULT`, `MULTU`	64-bit product, deposited in `HI:LO`
`DIV`, `DIVU`	32-bit quotient in `LO`, remainder in `HI`
`MFHI Rd`, `MFLO Rd`	move multiplier result registers to `Rd`

Immediate forms `ADDI`, `ADDIU`, `ANDI`, `ORI`, `XORI`, `SLTI`, and `SLTIU` follow the I-type format; the immediate is sign-extended for the arithmetic and comparison operations and zero-extended for the bitwise ones. `LUI Rt, imm` loads the immediate into the upper sixteen bits of `Rt`, clearing the lower sixteen, and is the standard means of constructing a 32-bit constant in two instructions.

There are no condition codes. All branches read general registers directly. We consider the avoidance of a global condition-code register to be one of the more important simplifications of the RISC approach; instructions remain free of an implicit dependency that would otherwise serialize the pipeline at every comparison.

The multiplier and divider produce results into the implicit `HI` and `LO` registers and proceed concurrently with the rest of the pipeline. Issuing a `MULT` or `DIV` while the previous one has not completed is permitted but stalls the issuing instruction until the prior result is read out of `HI` or `LO`. The reference implementation completes a 32×32 multiply in twelve cycles and a divide in thirty-five.

`ADD`, `SUB`, and `ADDI` raise an arithmetic-overflow trap on signed overflow; the unsigned-suffixed variants do not. Generated code that does not require overflow detection should use the unsigned forms.

6. Loads and Stores Through General Registers

Mnemonic	Effect
`LB Rt, offset(Rs)`	byte load, sign-extended
`LBU`	byte load, zero-extended
`LH`	halfword load, sign-extended
`LHU`	halfword load, zero-extended
`LW`	word load
`SB`, `SH`, `SW`	byte, halfword, and word stores

The effective address is `Rs + sign_extend(offset)`. Halfword and word addresses must be naturally aligned; misalignment raises a synchronous trap before the access proceeds. The address is translated through the current task’s page table; a TLB miss or a permission failure raises a synchronous trap routed to firmware.

Loads exhibit a one-cycle delay between the load and the use of its result. The reference implementation does not interlock on this hazard; the value of `Rt` in the cycle following the load is undefined

if read by the immediately succeeding instruction. The compiler is responsible for filling the load delay slot with an unrelated instruction or, in the absence of one, a `NOP` (canonically encoded as `SLL R0, R0, 0`).

7. Branches and Jumps

Mnemonic	Effect
`BEQ Rs, Rt, label`	branch if `Rs == Rt`
`BNE Rs, Rt, label`	branch if `Rs ≠ Rt`
`BLEZ Rs, label`	branch if `Rs ≤ 0` (signed)
`BGTZ Rs, label`	branch if `Rs > 0`
`BLTZ Rs, label`	branch if `Rs < 0`
`BGEZ Rs, label`	branch if `Rs ≥ 0`
`BLTZAL Rs, label`	`R31 ← PC+8`; branch if `Rs < 0`
`BGEZAL Rs, label`	`R31 ← PC+8`; branch if `Rs ≥ 0`
`J target`	unconditional jump
`JAL target`	`R31 ← PC+8`; jump to `target`
`JR Rs`	jump to `Rs`
`JALR Rd, Rs`	`Rd ← PC+8`; jump to `Rs`

`BLTZAL` and `BGEZAL` write the link register unconditionally and branch only if the condition holds; they are the architectural basis for PC-relative subroutine calls when `JAL`'s 256-megabyte reach is inadequate or undesirable.

All branches and jumps have a one-instruction delay slot: the instruction immediately following the branch executes regardless of whether the branch is taken. Code that cannot find a useful instruction to fill the slot inserts a `NOP`.

The branch displacement is the 16-bit `offset` field shifted left by two and added to the address of the delay-slot instruction. The reach of a conditional branch is therefore $\pm$128 KB; `J` and `JAL` reach a 256-megabyte region within the current address space.

8. Operations on Object Registers

The following operations move and inspect the contents of object registers without dereferencing them. They do not access memory through the object's storage; they read only the fields of the reference itself or the cached descriptor on the issuing processor. None of them requires the object to actually exist.

Mnemonic	Effect
`OMOV Od, Os`	`Od ← Os`
`ONULL Od`	`Od ← null`

Mnemonic	Effect
`OEQ Rd, Os, Ot`	`Rd ← 1` if `Os` and `Ot` denote the same object and generation, else `0`
`OISN Rd, Os`	`Rd ← 1` if `Os` is the null reference, else `0`
`OLEN Rd, Os`	`Rd ← length(Os)` in bytes
`OTAG Rd, Os`	`Rd ← type_tag(Os)`, zero-extended
`OHOME Rd, Os`	`Rd ← home_processor_id(Os)`
`OCAP Rd, Os`	`Rd ← capability_bits(Os)`

`OEQ` and `OISN` do not trap on null operands and may be used to test references unconditionally. The four inspection operations `OLEN`, `OTAG`, `OHOME`, `OCAP` raise a `null-dereference` trap if `Os` is the null reference; the descriptor lookup is otherwise free, completing in the same cycle as the comparison or arithmetic that consumes the result. A reference whose generation does not match the table entry raises `stale-reference` instead.

The capability bits, the home processor identifier, and the type tag are *readable* by user code but not modifiable. There is no instruction in this volume by which a user- or supervisor-mode program may construct a new object reference, alter the capability bits of an existing one, change the home processor of an object, or mutate any other field of the descriptor. Such operations are reached only through `CALL` to the appropriate firmware primitive.

9. Loads and Stores Through Object Registers

Mnemonic	Effect
`OLB Rt, offset(Os)`	byte load through object register
`OLBU`, `OLH`, `OLHU`, `OLW`	as for general-register loads
`OSB Rt, offset(Os)`	byte store through object register
`OSH`, `OSW`	halfword and word stores

The access covers `width` bytes starting at byte `offset` of the object referenced by `Os`. Three checks are performed in parallel with effective-address computation:

1. **Validity.** The reference must denote a live object: the generation counter recorded in the reference must match the generation in the object table. A mismatch raises `stale-reference`.
2. **Bounds.** Both `offset` and `offset + width` must lie within `[0, length(Os))`. A violation raises `bounds-violation`.
3. **Capability.** The capability bits of the reference must include `read` for a load and `write` for a store. A failure raises `capability-violation`.

Any failed check raises a synchronous trap before the memory access is issued. The faulting instruction is precise and the architectural state is exactly that of the cycle preceding it.

If the home processor of the object is the issuing processor, the access proceeds against local memory through the cache and memory hierarchy in the usual way. If the home is another processor,

the access is packaged as a crossbar message, routed to the home, executed there against local memory, and the response routed back; the issuing instruction stalls until the response returns. The architectural behaviour is identical in both cases — only the latency differs.

There is no architectural ordering guarantee between an object-register access and an ordinary load or store directed at the same byte through a mapping installed by firmware. Code that requires such ordering must either interpose an explicit firmware call or use the fence instruction reserved at `OBJECT` function code `0x8` for a future revision.

Object-register loads, like general-register loads, expose a one-cycle load-use delay to the compiler.

10. The SEND Instruction

```
SEND Os
```

`SEND` transmits a message to the home processor of the object referenced by `Os`. The message comprises:

- the recipient object reference (`Os`);
- the contents of `R4`–`R7` as a four-word integer payload;
- the contents of `O1`–`O4` as a four-reference object payload.

The capability bits of `Os` must include `send`; otherwise the instruction raises `capability-violation`. The reference must be live; a stale reference raises `stale-reference`. A null reference in `Os` raises `null-dereference`.

`SEND` is asynchronous. The instruction completes as soon as the crossbar accepts the message; it does not wait for the message to be delivered to a handler, and it produces no architectural reply. If the crossbar's outbound buffer for the destination port is full, the processor stalls until space is available, or — if the implementation elects to do so — raises `send-buffer-overflow` to firmware, which may then queue the message in software.

On the receiving processor, the firmware-installed handler for the target object is dispatched as if by an ordinary procedure call, with the integer and object payloads delivered in the same registers used to construct the message. The handler returns to firmware by `ERET`. If no handler is installed for the target object, the message is dropped and firmware is notified at its discretion.

`SEND` is the architectural primitive on which the system's higher- level communication patterns are built. Synchronous remote procedure call, actor-style message dispatch, and the rendezvous and broadcast patterns of the firmware itself all reduce to a sequence of `SEND`s combined with software-managed continuations and replies. A reply capability is itself an object reference passed in `O1`–`O4`, on which the recipient may in turn `SEND`.

11. The CALL Instruction

```
CALL #imm26
```

`CALL` traps the current task into firmware mode at the firmware entry vector for primitive number `imm26`. The architectural effect is:

1. `EPC` receives the address of the instruction following `CALL`.
2. The current privilege mode is recorded in the status register and the mode is changed to firmware.
3. Control transfers to the firmware entry vector indexed by `imm26`.

`CALL` does not have a branch delay slot. The instruction immediately following a `CALL` is the next user-visible instruction to execute on return from firmware; firmware is responsible for any pipeline cleanup required. This exception to the delay-slot convention is justified by the infrequency of `CALL` relative to ordinary control flow and by the substantial simplification it offers to firmware entry sequencing.

Arguments to a primitive are passed in `R4`–`R7` and `O1`–`O4`, following the standard calling convention; return values are returned in `R2`–`R3` and `O1`. Firmware preserves all caller-saved registers across the call. From the caller's perspective, `CALL` therefore behaves as an opaque procedure call with implementation-defined cost.

The set of valid primitive numbers, the meaning of each, and the trap conditions raised on invalid arguments are specified in Volume VI. A primitive number outside the range defined by the firmware implementation raises `reserved-call`, which firmware in turn handles by terminating the offending task or returning an error code at its discretion.

12. Privileged Instructions

The following instructions are valid only in supervisor or firmware mode. Execution in user mode raises `privileged-instruction`.

Mnemonic	Mode	Effect
`LCTRL Rd, ctrl`	sv / fw	read control register `ctrl` into `Rd`
`SCTRL ctrl, Rs`	sv / fw	write `Rs` into control register `ctrl`
`ERET`	sv / fw	return from exception or `CALL`
`WAIT`	sv / fw	halt processor until next interrupt
`TLBP`	fw only	probe TLB for entry matching `BadVAddr`
`TLBR`	fw only	read TLB entry indexed by `Index`
`TLBWI`	fw only	write the indexed TLB entry
`TLBWR`	fw only	write a TLB entry chosen at random

`LCTRL` and `SCTRL` access a numbered set of control registers including the status register, the exception program counter, the exception cause, the faulting address, and the page-table base. The detailed register set is enumerated in Volume V; the firmware-only registers controlling the object table, the crossbar routing, and the processor identification are not visible to supervisor-mode `LCTRL`.

`ERET` returns from the most recent exception or `CALL`: it restores the saved privilege mode and resumes execution at `EPC`. It takes no operand.

The TLB-management instructions are not normally invoked by hand- written code; firmware uses them in the trap handlers responsible for maintaining per-task page tables.

13. Traps, Exceptions, and the Restart Model

All exceptions on Object RISC are *precise*: when a trap handler is entered, every instruction earlier in program order than the faulting instruction has completed, and no instruction later in program order has any architectural effect. The address of the faulting instruction is captured in `EPC`; the cause is captured in `Cause`. After servicing the exception, firmware (or supervisor code, for the small number of supervisor-handled exceptions) returns by `ERET`.

The architecturally defined exceptions are:

Cause	Name	Description
`0x00`	`reset`	hardware reset
`0x01`	`external-interrupt`	masked by the status register
`0x02`	`bus-error-i`	instruction fetch failure
`0x03`	`bus-error-d`	data access failure
`0x04`	`address-misaligned-i`	misaligned program counter
`0x05`	`address-misaligned-d`	misaligned data access
`0x06`	`tlb-miss-i`	TLB miss on instruction fetch
`0x07`	`tlb-miss-d`	TLB miss on data access
`0x08`	`page-permission`	TLB hit but permission denied
`0x09`	`arithmetic-overflow`	trapping `ADD`, `SUB`, `ADDI`
`0x0a`	`reserved-instruction`	undecoded major opcode or `funct`
`0x0b`	`privileged-instruction`	privileged op in user mode
`0x0c`	`breakpoint`	reserved opcode for debuggers
`0x10`	`null-dereference`	use of `00` where forbidden
`0x11`	`stale-reference`	generation mismatch
`0x12`	`bounds-violation`	object-relative offset out of range
`0x13`	`capability-violation`	required capability bit absent
`0x14`	`send-buffer-overflow`	crossbar outbound buffer full
`0x20`	`firmware-call`	execution of `CALL`
`0x21`	`reserved-call`	`CALL` with primitive number out of range

The object-system traps `0x10`–`0x14` are routed to firmware regardless of the privilege mode at which the offending instruction was issued. Supervisor code that wishes to handle them on behalf of its tasks may do so by registering with firmware through the appropriate primitive.

External interrupts are masked individually and as a class through the status register. Their priority, numbering, and routing across the crossbar are specified in Volume IV.

A trap taken on an instruction in a branch delay slot records the address of the *branch* in `EPC` and sets the `BD` bit in the cause register. `ERET` resuming from such a trap re-executes the branch, which is the only way to ensure that the branch's effect on control flow is correctly preserved.

14. Reserved Encodings

This revision allocates thirty-three of the sixty-four major opcodes, twenty-two of the sixty-four `SPECIAL` function codes, and eight of the sixteen `OBJECT` function codes. Every unallocated encoding raises `reserved-instruction` when executed.

The following ranges are reserved for anticipated extensions; conforming implementations shall not allocate them to local additions:

- Major opcodes `0x10`–`0x1F` — floating-point and other coprocessor extensions, to be defined by a future revision.
- Major opcode `0x3E` — reserved for a future vector or wide-arithmetic extension.
- Major opcode `0x3F` — implementation-specific. Code using this opcode is not portable.
- `SPECIAL` function codes `0x30`–`0x3F` — reserved.
- `OBJECT` function code `0x8` — the explicit fence instruction promised in Section 9.
- `OBJECT` function codes `0x9`–`0xF` — reserved for further object- system primitives.

Appendix A — Major Opcode Map

The 64 major opcode slots are presently allocated as follows. – denotes a reserved encoding that raises `reserved-instruction`.

Opcode	Mnemonic	Opcode	Mnemonic
`0x00`	`SPECIAL`	`0x20`	`LB`
`0x01`	`REGIMM`	`0x21`	`LH`
`0x02`	`J`	`0x22`	—
`0x03`	`JAL`	`0x23`	`LW`
`0x04`	`BEQ`	`0x24`	`LBU`
`0x05`	`BNE`	`0x25`	`LHU`
`0x06`	`BLEZ`	`0x26`	—
`0x07`	`BGTZ`	`0x27`	—
`0x08`	`ADDI`	`0x28`	`SB`
`0x09`	`ADDIU`	`0x29`	`SH`
`0x0A`	`SLTI`	`0x2A`	—
`0x0B`	`SLTIU`	`0x2B`	`SW`
`0x0C`	`ANDI`	`0x2C`–`0x2F`	—
`0x0D`	`ORI`	`0x30`	`OBJECT`
`0x0E`	`XORI`	`0x31`	`OLB`
`0x0F`	`LUI`	`0x32`	`OLH`
`0x10`–`0x1F`	reserved (FP)	`0x33`	`OLW`

Opcode	Mnemonic	Opcode	Mnemonic
		0x34	OLBU
		0x35	OLHU
		0x36	—
		0x37	—
		0x38	OSB
		0x39	OSH
		0x3A	—
		0x3B	OSW
		0x3C	SEND
		0x3D	CALL
		0x3E	reserved (vec)
		0x3F	implementation

The REGIMM major opcode at 0x01 is decoded by the contents of the rt field, encoding the conditional branches BLTZ, BGEZ, BLTZAL, and BGEZAL. The SPECIAL and OBJECT major opcodes are decoded by their funct fields as described in Sections 5 and 8 respectively.

Appendix B — Trap Vector Layout

Each architectural exception has a fixed entry point in firmware, expressed as an offset from the firmware vector base register (VECBASE, a firmware-only control register). The offsets are sixty-four bytes apart, sufficient for a short trampoline at each entry; firmware that requires more space at a particular vector branches out to a longer handler from within the trampoline.

Cause	Offset	Cause	Offset
0x00	0x000	0x10	0x400
0x01	0x040	0x11	0x440
0x02	0x080	0x12	0x480
0x03	0x0C0	0x13	0x4C0
0x04	0x100	0x14	0x500
0x05	0x140	0x20	0x800
0x06	0x180	0x21	0x840
0x07	0x1C0		
0x08	0x200		
0x09	0x240		
0x0a	0x280		
0x0b	0x2C0		
0x0c	0x300		

The reset vector at offset `0x000` is entered with the processor in firmware mode and all interrupts masked. Firmware is responsible for initializing the remaining control registers, the object descriptor cache, and the connection to the crossbar before transferring control to the supervisor.

Volume III — Object System

1. Scope

This volume specifies the data structures and invariants of the Object RISC object system: the layout of an object reference, the format of the object descriptor, the per-processor object table, the descriptor cache on each processor, the meaning of the capability bits, the lifecycle of an object from allocation through revocation, and the firmware-mediated mechanism by which objects migrate between processors.

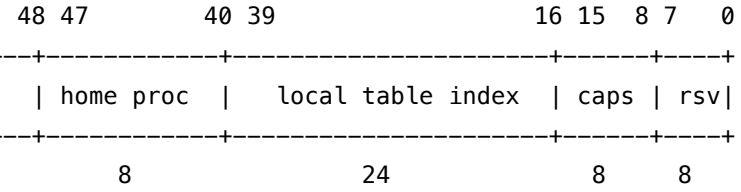
The instructions that operate on this machinery — `OL*`, `OS*`, `OMOV`, `OEQ`, `OISN`, `OLEN`, `OTAG`, `OHOME`, `OCAP`, `SEND`, and `CALL` — are defined in Volume II. The wire-level messages by which descriptor fetches, invalidations, and forwardings travel between processors are defined in Volume IV. The firmware primitives invoked through `CALL` to allocate, revoke, derive, migrate, and map objects are catalogued in Volume VI; this volume names them but does not specify their detailed parameter conventions.

The reader is assumed to be familiar with Volumes I and II.

2. Object References

2.1 Reference Format

An object reference is the 64-bit value held in an object register. It is structured as four contiguous fields:



- **Generation** (16 bits, unsigned). The value of the generation counter at the home processor's object table at the moment this reference was minted. It increments each time the underlying table slot is freed; on access, the reference's generation must match the generation currently recorded in the descriptor, or the reference is *stale*.
- **Home processor** (8 bits, unsigned). The identifier of the processor whose object table contains the descriptor for this object. Up to 256 processors are addressable in a single Object RISC system; larger systems are reached through the routing extensions of Volume IV.

- **Local table index** (24 bits, unsigned). The position within the home processor's object table at which the descriptor resides. Up to 16,777,216 objects may be resident on any one processor, an allocation we judge generous against the physical-memory sizes of the late 1980s.
- **Effective capability bits** (8 bits). The capability rights this particular reference carries. The bits are checked by hardware on every dereference without recourse to the descriptor (Section 5). They may be no stronger than the *maximum* capability bits recorded in the descriptor, an invariant that firmware maintains by construction at every primitive that produces a reference.
- **Reserved** (8 bits). Must be zero in this revision. Future revisions may use these bits to encode access hints or, in a wider reference format, to extend the local table index.

A reference is *opaque* in the sense that user-mode and supervisor-mode code cannot construct one whose generation, home processor, or table index is selected freely; the only operations available to non-firmware code are to read these fields through `OHOME` and `OTAG` (and other inspection ops in Volume II), to copy a reference from one object register to another through `OMOV`, and to weaken its capability bits through the firmware primitive `ObjDerive`. The capability bits, the home processor, and the index are visible to any holder of the reference; nothing about them is secret. The integrity of the system rests not on the obscurity of references but on the impossibility of manufacturing one with rights that exceed those of an existing reference.

2.2 The Null Reference

The constant value zero in all fields is the *null reference*, hardwired into object register `O0`. Its generation, home, and index fields are by construction invalid, and no live object ever has them. The null reference dereferences to the synchronous trap `null-dereference`; it carries no capability bits and cannot be strengthened.

`OEQ` and `OISN` test the null reference without trapping.

2.3 Reference Equality

Two references denote the same object only if they share the same generation, home processor, and local table index. The capability and reserved fields are *not* significant for equality; references derived from a common ancestor through `ObjDerive` with different capability masks compare equal under `OEQ`.

A pair of references whose triples agree but whose generation differs do *not* denote the same object. They denote different generations of the same table slot, which by construction are different objects; this is the case `OEQ` returns zero, even though it may appear at first glance that two "almost equal" references should compare equal. In practice this case is rare; the generation counter exists precisely to make it always discoverable.

3. The Object Table

3.1 Per-Processor Layout

Each processor maintains an object table holding the descriptors of every object whose home it is. The table is a contiguous array of fixed-size descriptors, allocated by firmware in the local physical memory at boot, and pinned for the duration of the system's operation.

The table's base physical address is held in the firmware-only control register `OBJTAB_BASE`; its size, in descriptor entries, is held in `OBJTAB_LIMIT`. A reference whose local index exceeds `OBJTAB_LIMIT` on its home processor is trivially invalid and raises `stale-reference` on dereference.

Allocation of fresh entries within the table is a firmware concern; the architecture neither specifies nor restricts the allocation discipline. The reference implementation maintains a free list threaded through the unused descriptors of the local table.

3.2 Descriptor Format

Each descriptor occupies thirty-two bytes, naturally aligned, and is structured as follows:

```
Offset  Size  Field
+0      4     base — physical base address (on home processor)
+4      4     length — size of the object in bytes
+8      2     generation — current generation counter for this slot
+10     2     type_tag — opaque software-defined type identifier
+12     1     max_caps — maximum capability bits ever issuable
+13     1     flags — descriptor state (see Section 3.3)
+14     2     reserved — must be zero
+16     8     send_handler_ref — object reference of the handler code
+24     4     send_handler_off — byte offset within the handler object
+28     4     reserved — must be zero
```

The fields are interpreted as follows.

- **base** is the physical byte address, on the home processor, at which the object's storage begins. The storage region is contiguous and lies entirely within the home processor's local memory.
- **length** is the size of the storage region in bytes. The legal byte offsets into the object are `[0, length)`.
- **generation** is incremented each time the slot is freed. A reference is live if and only if its generation field equals this value.
- **type_tag** is an opaque 16-bit value supplied by the allocator at `ObjAlloc` time. The hardware does not interpret it. Software conventions assign meaning to ranges of values.
- **max_caps** is the upper bound on the capability bits any reference to this object may carry. `ObjDerive` may produce references with any subset of `max_caps`; no primitive produces a reference with a bit set in its caps field that is not also set in `max_caps`.
- **flags** carries descriptor state, encoded as Section 3.3 details.

- **send_handler_ref** and **send_handler_off** together name the code invoked when a `SEND` message arrives addressed to this object.

The two reserved fields are written as zero by firmware on allocation; future revisions may consume them for additional state without changing the descriptor's size.

3.3 Descriptor Flags

The `flags` byte encodes the descriptor's lifecycle state:

Bit	Name	Meaning
0	`LIVE`	Slot holds a valid object
1	`MIGRATING`	Migration is in progress (Section 6.4)
2	`FORWARDED`	This slot is a stub forwarding to elsewhere
3	`PINNED`	Storage may not be moved or paged
4	`DEVICE`	Object overlays a device register window
5	`EXECUTABLE`	Storage holds executable code
6–7	reserved	Must be zero

A `FORWARDED` descriptor reinterprets its `base` and `length` fields as the new home processor and new local index, respectively, of the object's true location; see Section 6.4.

4. The Object Descriptor Cache

Every dereference of an object register requires the descriptor of the named object: at minimum, the generation (for liveness), the length (for bounds), and the base address (to locate the storage). Fetching the descriptor from the home processor's object table on every access would impose a long-latency dependency on every object- register load and store, defeating the design goal of zero-cycle overhead in the common case.

The architecture therefore requires every implementation to maintain an *object descriptor cache* (ODC), a small associative structure local to the processor that holds recently-used descriptors. The contents of the ODC are coherent with the object table by the mechanism of Section 4.2; programs that observe the ODC's behaviour only through `OL*`, `OS*`, and the inspection operations of Section 8 of Volume II cannot distinguish it from a hypothetical implementation that performed a fresh descriptor fetch on every access.

4.1 Tag, Lookup, and Hit Path

An ODC entry is keyed by the tuple `(home, index)` taken from the reference's home and table-index fields. On the issuing of an object- register access, the ODC is consulted in the same cycle as the effective-address computation. A hit returns the entry's cached descriptor; the bounds check and the dispatch of the access (local versus remote) proceed without further dependency. A miss stalls

the issuing instruction and dispatches a `DESC_REQ` message to the home processor, which responds with a `DESC_RESP` message carrying the descriptor; the entry is installed and the access retried.

The reference implementation provides a thirty-two-entry, four-way set-associative ODC; conforming implementations may vary the size and associativity, but are required to provide an ODC of at least eight entries.

4.2 Generation as Coherence Mechanism

The ODC is kept consistent with the object table by the generation counter rather than by explicit invalidation traffic. On every hit, the hardware compares the generation field of the *reference being dereferenced* against the generation field of the *cached descriptor*. If they disagree, the entry is treated as a miss, evicted, and refetched. Code that holds a stale reference therefore sees its access fail at the descriptor cache itself; code that has obtained a fresh reference after the slot was reused sees the new descriptor on its first access.

This mechanism removes the need for cross-processor invalidation broadcasts on every `ObjFree`, the cost of which would dominate any busy system. It does, however, mean that the descriptor's *non-generation* fields — most importantly the maximum capability bits, the type tag, and the send handler — must not be modified without also incrementing the generation. Consequently, those fields are immutable in this revision: changing them requires freeing the object and allocating a new one. The send handler is the sole exception; it is described in Section 7.

The generation field of the descriptor wraps after 65,536 increments. A reference whose generation has been overrun is technically indistinguishable from a reference to the current incarnation; the firmware specifications of Volume VI are required to recycle table slots in a manner that makes wrap-around in practice unobservable to correctly written software (typically by keeping a slot retired for many generations after `ObjFree`, or by allocating from the slot least-recently freed). The architecture imposes no specific policy.

5. Capability Bits

5.1 The Capability Set

The eight capability bits are assigned as follows:

Bit	Symbol	Right
0	`R`	Read storage through `OL*`
1	`W`	Write storage through `OS*`
2	`X`	Treat storage as executable for an indirect jump
3	`S`	Issue `SEND` to this object
4	`V`	Revoke this object through `ObjRevoke` or `ObjFree`
5	`M`	Request migration of this object
6	`C`	Derive further references from this one
7	reserved	Must be zero in this revision

The hardware enforces `R`, `W`, and `S` at access; `X` is enforced by the page-permission machinery of Volume V at the point an executable mapping is established; `V`, `M`, and `C` are enforced by firmware at the relevant primitive call.

A reference with no capability bits set is *inert*: it can be moved, compared, and inspected, but no useful operation can be performed through it. Inert references are the canonical handle to an object held purely for naming purposes — for example, an entry in a software-managed table of known objects.

5.2 Derivation

The firmware primitive `ObjDerive(ref, mask) → ref'` produces a new reference to the same object with effective capabilities equal to `ref.caps ∧ mask`. Derivation requires that the calling reference's `C` bit is set; firmware refuses the primitive otherwise. The derived reference inherits the same generation, home, and index, and is indistinguishable from the original under `OEQ`.

There is no operation that strengthens capabilities. The architecture treats `ObjDerive` as the only producer of reference values from existing references, and no other primitive returns a reference whose capabilities are not bounded above by those of the inputs from which it was derived.

5.3 Revocation

Revocation invalidates every outstanding reference to an object in a single firmware-mediated step. The firmware primitive `ObjRevoke(ref)` requires the `V` bit on the calling reference and increments the generation counter of the descriptor without freeing the underlying storage; subsequent dereferences of any outstanding reference to the revoked object see a generation mismatch and fail at `stale-reference`.

Revocation is the architecture's primitive answer to the question of how rights granted to one program may be withdrawn by another. The holder of a reference with `V` may revoke; the revocation is global, because no separate state per holder is maintained. This coarseness is intentional: tracking individual outstanding references for fine-grained revocation would impose either a system-wide reverse-index of references (prohibitively expensive) or a software discipline that the architecture chooses not to mandate.

`ObjFree(ref)` is `ObjRevoke` followed by the release of the storage back to the home processor's free pool. It also requires the `V` bit.

6. Object Lifecycle

6.1 Allocation

`ObjAlloc(length, type_tag, init_caps) → ref` creates a new object on the calling processor (the home is fixed by the call site; objects may not be allocated remotely). Firmware:

1. Selects an unused slot from the local object table's free list.
2. Allocates a contiguous physical region of `length` bytes from the local memory pool.

3. Zeroes the storage.
4. Constructs a descriptor with the chosen base, the requested length and type tag, `max_caps` set to `init_caps`, the generation taken from the slot's current value, and `flags` set to `LIVE`.
5. Constructs a reference whose generation, home, and index match the descriptor, and whose effective capabilities equal `init_caps`.

The returned reference has `max_caps` worth of rights; subsequent holders may receive references derived to weaker rights but never stronger.

If the local memory pool cannot satisfy the request, firmware returns a null reference and an error code per Volume VI's call convention. The architecture provides no hardware-level mechanism for cross- processor allocation: a task that wishes to create an object on a different processor must `SEND` a request to a service object on that processor, whose handler issues the local `ObjAlloc` and returns the new reference.

6.2 Deallocation

`ObjFree(ref)` increments the descriptor's generation, returns the storage to the local free pool, and threads the table slot back into the free list. Outstanding references become stale immediately; their next dereference fails at the descriptor cache, regardless of which processor holds them.

Storage returned to the free pool may be reused for any subsequent allocation, including allocations of objects with different lengths and capabilities. The generation increment is the sole guarantor that old references do not accidentally name the new object.

6.3 Derivation

See Section 5.2.

6.4 Migration

Migration moves the storage of an object from one home processor to another without invalidating any outstanding reference. The firmware primitive `ObjMigrate(ref, new_home)` requires the `M` bit on the calling reference. The protocol is:

1. The source firmware sets the `MIGRATING` flag on the descriptor at the original home.
2. The source firmware copies the storage to the destination processor, and the destination firmware allocates a fresh slot in its local table holding a descriptor with the same generation, the same `max_caps`, the same `type_tag`, the new physical base, the same length, and `flags` set to `LIVE`.
3. The source firmware replaces the original descriptor in place with a *forwarded* descriptor: `flags` is set to `FORWARDED`, and the `base` and `length` fields are reinterpreted as the new home and new local index, respectively.
4. The source firmware retains the slot in its forwarded state. It is not returned to the free list.

Subsequent dereferences of references that still name the original home arrive at the forwarded descriptor. The home processor's firmware responds with a `DESC_FORWARD` crossbar message that

carries both the redirection and the new descriptor; the issuing processor installs the new descriptor in its ODC, *under the original `(home, index)` tag*, and the original reference therefore continues to function but with all subsequent accesses dispatched directly to the new home. The forwarded slot at the source remains until firmware chooses to garbage-collect it, by which time the architecture is silent.

Migration does not increment the generation. A reference that was live before migration remains live after.

6.5 Mapping

`MapObject(ref, va_hint, offset, length, prot) → va` installs page- table entries in the calling task's address space that resolve `va` through `va + length` to the storage of the object referenced by `ref`, beginning at the given byte offset. The calling task may then access the storage through ordinary loads and stores in addition to, or in place of, object-register dereferences. The `prot` parameter is required to be a subset of the reference's effective capabilities restricted to `R`, `W`, and `X`.

Mapping is restricted to objects whose home is the local processor, as stated in Volume I, Section 4. Attempting to map a remote object returns an error to the caller; the firmware primitive `ObjMigrate` must be invoked first if local mapping is desired.

The detailed parameter conventions of `MapObject`, `Unmap`, and `Protect` are given in Volume VI, Section 5.

7. Send Handlers

7.1 Installation

The `send_handler_ref` and `send_handler_off` fields of the descriptor name the code that runs when a `SEND` message arrives addressed to this object. The two fields together specify an entry point: the code begins at byte `send_handler_off` of the object referenced by `send_handler_ref`, which must carry the `X` capability.

These fields are populated by the firmware primitive `InstallHandler(target_ref, handler_ref, handler_off)`. The primitive does not increment the descriptor's generation; the send-handler fields are the only mutable parts of the descriptor and are excluded from the immutability invariant of Section 4.2 because they are read only by the home processor's firmware on receipt of a `SEND`, and the read is performed against the live descriptor without any caching.

7.2 Dispatch

When a `SEND_DELIVER` message arrives at a processor, firmware on the receiving side:

1. Validates the recipient reference: the home in the delivered ref must be this processor, the index must be in range, the generation must match.
2. Reads `send_handler_ref` and `send_handler_off` from the descriptor.

3. Constructs a fresh task (or reuses a worker from a pool) and transfers control to the handler entry point. The integer payload of the SEND is installed verbatim in `R4`–`R7`. The object payload, which on the wire carries four references, is delivered to the handler asymmetrically:
   - `O1` receives a freshly-constructed, full-capability reference to the recipient object itself — the handler's *self* reference. This overrides what the first wire-format object payload slot would otherwise have carried.
   - `O2`, `O3`, `O4` receive the *first*, *second*, and *third* object references from the SEND payload as transmitted on the wire, respectively.
   - The *fourth* reference of the SEND payload is delivered to the handler's task control side-channel buffer (Volume VI Section 11) and is accessible through the firmware primitive `MessagePayloadOR4` for handlers that require it.

This dispatch convention applies only to handler invocation. Receive queues established by `ReceiveQueueAttach` (Volume VI Section 6) deliver the SEND payload verbatim to the polling task: all four wire-format object references appear unmodified in `O1`–`O4` of the polling task on return from `ReceiveQueuePoll`, and the self-reference convention does not apply.

If no handler is installed (the `send_handler_ref` is null), the message is dropped and an event is posted to the firmware diagnostic log. The architecture does not require the message be retried, queued, or acknowledged in this case; recovery is a software responsibility above the level of this volume.

8. Reserved Fields and Future Extensions

The following fields are reserved in this revision and shall be written as zero by all conforming firmware:

- Bits 7:0 of an object reference (the `rsv` field).
- Bit 7 of the capability byte (the spare capability bit).
- Bits 6 and 7 of the descriptor's `flags` byte.
- Bytes 14:15 and 28:31 of the descriptor.

Future revisions are expected to consume some of this space for the following purposes, listed without commitment:

- An additional capability bit naming the right to install send handlers, separating it from the present `V` bit.
- A descriptor flag distinguishing read-mostly from write-mostly objects, as a hint to the cache-replacement policy.
- A short generation extension widening the counter from sixteen to twenty-four or thirty-two bits, addressing the wrap-around concern noted in Section 4.2 at the cost of a wider reference format.

Volume IV — Interconnect Protocol

1. Scope

This volume specifies the wire-level protocol of the Object RISC crossbar interconnect: the physical packet format, the set of message types, the arbitration discipline at each output port, the routing of packets across single and hierarchical crossbar configurations, the flow-control regime by which sources and sinks coordinate, the timing and clock-distribution requirements for a conforming implementation, and the small set of error conditions the protocol recognizes.

It does not specify the layout of the silicon that implements the protocol; the OR-XBAR-1 reference crossbar is described in Volume V. It also does not specify the firmware that interprets the higher-level semantics carried by these messages; that is the subject of Volume VI.

The reader is assumed to be familiar with Volumes I, II, and III.

2. Topology

An Object RISC system consists of one or more *processors*, each identified by an 8-bit *processor identifier* assigned at boot. Every processor is a port of the crossbar; the crossbar has no other class of port. Memory modules are local to each processor and are reached across the crossbar only by way of that processor's local memory controller.

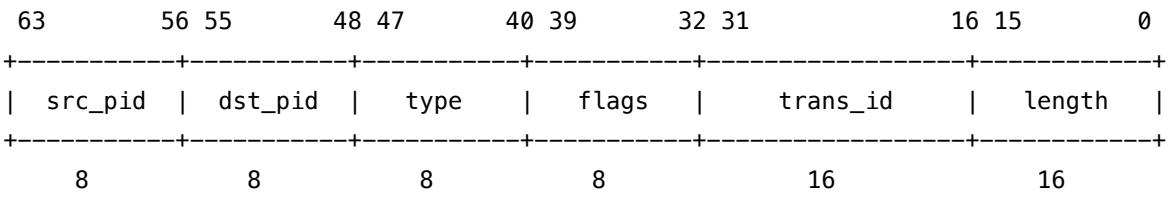
The reference configuration is a single 16-port crossbar interconnecting sixteen processors. The protocol admits configurations of one to sixteen ports per physical crossbar chip and, by hierarchical arrangement, up to 256 processors total. The detailed packaging of larger systems is left to the integrator; the protocol is concerned only with the contract on the wire.

Each port presents to its processor a *transmit channel* and a *receive channel*, each carrying one packet word per cycle. The width of a packet word is 32 bits. The two channels operate independently and may be driven in the same cycle.

3. Packet Format

3.1 Header

Every packet begins with a single 64-bit header word (transmitted as two 32-bit cycles, most significant first) of the following layout:



- **src_pid** (8 bits). The processor identifier of the originating port. Receivers use this field to direct responses and to credit flow control.
- **dst_pid** (8 bits). The processor identifier of the destination port. In a single-crossbar configuration, this is the port number; in a hierarchical configuration, it is interpreted by the routing table at each intermediate hop (Section 6).
- **type** (8 bits). Selects the message format from the table of Section 4.
- **flags** (8 bits). Per-message flags. The bits assigned in this revision are described per type; bits 7:6 are reserved at the protocol level for `ACK_REQ` and `LAST_FRAGMENT`, which apply uniformly to every message type.
- **trans_id** (16 bits). A per-source transaction identifier, selected by the source. The destination uses this field to correlate request and response messages; the value is opaque to intermediate hops.
- **length** (16 bits). The length of the payload, in 32-bit words. Zero is permitted and indicates a header-only message. The maximum payload of 65,535 words (256 KB) is more generous than any single message in this revision uses; longer transfers are decomposed into multiple packets in software.

3.2 Payload Encoding

The payload immediately follows the header and consists of `length` contiguous 32-bit words. Each message type specifies the layout of its payload words; some types have variable payloads (notably the data- carrying responses), in which case the layout is also defined per type.

A single trailing word, the *checksum*, follows the payload. It carries a 32-bit word formed by the bitwise exclusive-OR of every header and payload word transmitted in the packet. A receiver that observes a checksum mismatch raises `LINK_ERROR` to firmware and discards the packet.

The checksum is not included in the `length` field; a packet's total on-wire length is `2 + length + 1` words.

4. Message Types

The 256 type codes are partitioned by purpose. Codes `0x10`–`0x1F` are the object memory-access messages, `0x20`–`0x2F` the inter-task communication messages, `0x30`–`0x3F` the descriptor maintenance messages, and `0xF0`–`0xFF` the link-control messages. The remaining code ranges are reserved for future use.

4.1 Object Memory Access

These messages implement the dispatch of `OL*` and `OS*` instructions issued through references whose home is a different processor.

Code	Name	Direction	Payload
`0x10`	`OBJ_READ_REQ`	issuer → home	4 words: ref low, ref high, offset, width
`0x11`	`OBJ_READ_RESP`	home → issuer	`ceil(width/4)` words of data, byte-padded
`0x12`	`OBJ_WRITE_REQ`	issuer → home	4 + `ceil(width/4)` words: ref, offset, width, data
`0x13`	`OBJ_WRITE_RESP`	home → issuer	0 words; flags carry success or fault code

The reference is transmitted little-end first to align with the layout of the issuing processor's pair-register pair. The home processor performs the validity, bounds, and capability checks itself: the issuing processor has already confirmed the cached descriptor's caps match, but the home is the authoritative source and may discover that the object has been freed (generation mismatch) or that the descriptor in the issuing processor's cache was stale. In any such case the response carries a fault code in `flags` rather than data.

The fault codes carried in `flags` for the response messages are:

Bits 5:0	Name	Meaning
`0x00`	`OK`	Access succeeded
`0x01`	`STALE`	Generation mismatch at home
`0x02`	`BOUNDS`	Offset out of range
`0x03`	`CAP`	Capability denied
`0x04`	`BUS_ERROR`	Underlying memory error
`0x3F`	`FORWARDED`	Object has migrated; see DESC_FORWARD

`OBJ_READ_RESP` carries no data words when its flags indicate any fault other than `OK`; the issuing processor raises the corresponding synchronous trap on the original instruction.

4.2 Inter-Task Communication

Code	Name	Direction	Payload
`0x20`	`SEND_DELIVER`	issuer → home	14 words: 2 ref, 4 GPR, 8 OR

`SEND_DELIVER` is the wire form of the `SEND` instruction. Its payload comprises:

- two words for the recipient object reference;
- four words for the integer payload (`R4`–`R7`);
- eight words for the four-reference object payload (`O1`–`O4`).

The recipient verifies the reference at delivery, dispatches to the installed handler, and produces no response packet on success. If the recipient is invalid (generation mismatch, no installed handler, or home not this processor), the firmware diagnostic log is appended and no further action is taken; the architecture does not require a NAK.

The mapping of the eight payload words for object references onto the handler's object register file is asymmetric: the wire transports four references, but the handler receives only three of them in `O2`–`O4`, with `O1` overridden by a fresh self-reference and the fourth wire slot held in the side-channel buffer. The full convention is given in Volume III Section 7.2 and the corresponding firmware primitive in Volume VI Section 6.

`SEND_DELIVER` carries the `ACK_REQ` flag at the discretion of higher- level software. When set, the recipient sends a `SEND_ACK` (`0x21`) header-only message back to the source on completion of dispatch. The ACK does not indicate that the handler has run to completion, only that it has been entered.

4.3 Descriptor Maintenance

These messages keep each processor's object descriptor cache coherent with the object table at the home processor.

Code	Name	Direction	Payload
`0x30`	`DESC_REQ`	issuer → home	1 word: index
`0x31`	`DESC_RESP`	home → issuer	8 words: descriptor
`0x32`	`DESC_FORWARD`	home → issuer	8 words: new descriptor (with new home and index)
`0x33`	`DESC_INVALIDATE`	home → all (broadcast)	1 word: index

`DESC_REQ` is issued by an issuing processor on a miss in its object descriptor cache; the home responds with the live descriptor in a `DESC_RESP`. If the home's descriptor is `FORWARDED`, the response type is instead `DESC_FORWARD`, and the issuing processor installs the descriptor under the *original* `(home, index)` tag of the requested object, redirecting subsequent accesses to the migrated location while preserving the validity of the original reference.

`DESC_INVALIDATE` is issued by a home processor when it has reason to believe one or more issuing processors hold a stale entry for an object whose state has changed in a manner not captured by the generation counter — for example, when the send handler is updated. In the present revision the only field that may be modified without a generation increment is the send handler, which is read only at the home and therefore never cached at issuing processors; `DESC_INVALIDATE` is consequently reserved but unused. It is specified here against future extensions that might cache further descriptor fields at the issuing side.

4.4 Link Control

Code	Name	Direction	Payload
`0xF0`	`LINK_FLOW_CREDIT`	port → port	1 word: credits
`0xF1`	`LINK_ERROR`	port → firmware	1 word: cause
`0xFE`	`LINK_HEARTBEAT`	port → port	0 words
`0xFF`	`LINK_RESET`	port → all	0 words

Link-control messages are not delivered to any processor's user- visible interface; they are consumed by the crossbar interface hardware on the receiving port. The single exception is `LINK_ERROR`, which is forwarded to firmware on the receiving processor for diagnostic logging.

`LINK_HEARTBEAT` is exchanged at intervals defined by the implementation (typically every 4,096 cycles) to confirm liveness; the absence of expected heartbeats is reported through `LINK_ERROR`.

`LINK_RESET` is broadcast on power-up and on firmware request; it returns every input FIFO and every flow-control counter to its initial state, and is the only message permitted to bypass arbitration.

5. Arbitration

Each output port arbitrates among the input ports requesting it in any given cycle. The arbitration discipline is *rotating priority*: the output port maintains a single 8-bit register, the priority pointer, which selects the input port that, among those requesting in the current cycle, is granted access. The pointer advances by one position modulo the port count after each grant.

This discipline guarantees that no requesting input is starved for more than `N - 1` cycles, where `N` is the port count, regardless of the demand pattern. It is also synthesizable in a single cycle for the reference 16-port configuration in 1.5-micron CMOS, which is the underlying constraint motivating its choice.

Arbitration is performed independently at each output port. There is no global serialization of grants; up to `N` packets may begin transmission in the same cycle, provided each has a distinct output port.

6. Routing

6.1 Single-Crossbar Configuration

In a single-crossbar configuration the destination processor identifier in the packet header is the physical output port number. The crossbar inspects only this field and the source identifier; no routing table is consulted, and no per-hop state is maintained.

The single-crossbar configuration is the configuration of any system of sixteen processors or fewer. We expect it to remain the dominant configuration for the lifetime of this revision.

6.2 Hierarchical Configuration

Configurations of more than sixteen processors are constructed by arranging multiple crossbar chips in a hierarchy: a *root* crossbar whose ports each connect to a *leaf* crossbar, with processors attached to the leaves. The reference scaling, which permits up to 256 processors, is a single root with sixteen leaves of sixteen ports each, with one port on each leaf consumed by the connection to the root.

In a hierarchical configuration, every port of every crossbar chip carries a *routing table* of 256 entries, indexed by the destination processor identifier and producing a 4-bit *next-hop port number*. The table is loaded by firmware at boot and is invariant during operation. A packet arriving at a port whose routing table indicates its own number is delivered to the local processor (for a leaf) or discarded with a `LINK_ERROR` (for any other case, indicating a configuration error).

Larger configurations than the 256-processor reference scaling are permitted, but require widening the destination identifier field beyond 8 bits and consequently the routing table; such extensions are deferred to a future revision.

7. Flow Control

Each input port maintains a fixed-size FIFO of 32 words. Flow control is credit-based: the source maintains a counter of available credits for each destination it transmits to, decrementing the counter by the length of each transmitted packet (including header and checksum). The destination periodically returns a `LINK_FLOW_CREDIT` message indicating credits liberated by the FIFO's drainage.

A source whose credit counter is too low to admit a packet stalls the packet's transmission until a `LINK_FLOW_CREDIT` arrives. The processor's crossbar interface presents the stall to the issuing instruction as a memory stall indistinguishable from a long-latency cache miss; the architecture observably reflects flow-control stalls only in the cycle counts, not in the program-visible state.

The 32-word input FIFO is sized to absorb at least one full `SEND_DELIVER` packet (17 words including header and checksum) plus one full `OBJ_WRITE_REQ` packet of moderate width; conforming implementations may size FIFOs more generously to reduce flow-control round trips, at the obvious cost in silicon area.

8. Timing and Synchronization

The reference crossbar is fully synchronous: every port samples its input on the rising edge of a single distributed clock, and every arbitration decision is taken in the same cycle as the corresponding port-grant. Implementations may choose between a directly distributed clock and a phase-locked loop per port, but the architecture requires that the maximum cycle-to-cycle skew between any two ports of the same crossbar chip not exceed one nanosecond at the reference 20 MHz clock rate.

Inter-port latency at a single crossbar chip is two cycles plus the packet length: one cycle for arbitration, one cycle to traverse the switching matrix, and one cycle per word transmitted thereafter.

In the reference 16-port configuration at 20 MHz, a four-word `OBJ_READ_REQ` therefore consumes 6 cycles or 300 nanoseconds at the crossbar itself; the round-trip latency from issue to data return, including dispatch at the home processor, is approximately 30 cycles.

In a hierarchical configuration each additional hop adds one cycle of arbitration and one cycle of traverse, plus any flow-control stall.

9. Error Detection and Recovery

The crossbar protocol recognizes a small set of error conditions, all of which are reported via `LINK_ERROR` to firmware on the receiving port for diagnostic logging. The architecture does not require hardware-level retry; recovery from any error is a firmware policy.

Cause word	Name	Source
`0x01`	`CHECKSUM_MISMATCH`	Receiver, on bad packet
`0x02`	`RESERVED_TYPE`	Receiver, on undefined message type
`0x03`	`MISDIRECTED`	Crossbar, on routing-table miss
`0x04`	`FIFO_OVERFLOW`	Receiver, on credit violation
`0x05`	`HEARTBEAT_LOST`	Receiver, on missed heartbeat
`0x06`	`RESET_OBSERVED`	Receiver, after `LINK_RESET`

A receiver that detects `CHECKSUM_MISMATCH` discards the packet and does not reply; the source learns of the loss only by way of an absent response (for request/response message pairs) or not at all (for fire-and-forget messages such as `SEND_DELIVER`). Higher-level recovery, including timeouts and retransmission, is a software responsibility.

10. Reserved Fields and Future Extensions

The following encodings are reserved in this revision and shall not be used by conforming implementations:

- Type codes `0x00`–`0x0F`, `0x14`–`0x1F`, `0x22`–`0x2F`, `0x34`–`0xEF`, `0xF2`–`0xFD` (every code not enumerated in Section 4).
- Bits 5:0 of the header `flags` byte for non-response messages.
- Bits 4 and 3 of the response fault-code field.

Anticipated extensions that have already been provisionally allocated:

- An optional broadcast message type, `0x40`, for system-wide notifications such as global time advancement.
- A descriptor-cache-fill prefetch hint, `0x35`, by which firmware on one processor may speculatively warm the descriptor cache of another.
- A widened destination identifier for systems of more than 256 processors, requiring a different header layout and consequently a different protocol revision number.

Volume V — Reference Implementation

1. Scope

This volume describes a representative implementation of the Object RISC architecture: the OR-1000 processor and the OR-XBAR-1 crossbar. The implementation is offered as an existence proof that a useful Object RISC system can be constructed in 1.5-micron CMOS at 1986 transistor budgets, and as a yardstick against which alternative implementations may be measured. Conforming implementations are not required to match it in microarchitecture, only in the architectural contracts of Volumes I through IV and the firmware interface of Volume VI.

The volume covers, in order: the OR-1000 pipeline; its caches, TLB, and object descriptor cache; the multiplier and divider; the external bus and crossbar interface; the architecturally visible control register set; the OR-XBAR-1 crossbar's port organization, arbitration, and buffering; representative system configurations from a single-processor workstation to a 16-processor server; the reset and boot sequence; the physical characteristics of both chips; and a small body of performance estimates.

The reader is assumed to be familiar with Volumes I through IV.

2. The OR-1000 Processor

2.1 Specifications

Parameter	Value
Process	1.5-micron, two-layer-metal CMOS
Transistors	approximately 110,000
Die size	9.8 mm × 9.8 mm
Pin count	168 (PGA)
Clock	16 MHz (commercial), 20 MHz (selected)
Power dissipation	2.4 W typical at 16 MHz, 5 V supply
Pipeline	5-stage, single-issue, in-order
Instruction cache	4 KB, direct-mapped, 16-byte lines
Data cache	4 KB, direct-mapped, 16-byte lines
Object descriptor cache	32 entries, 4-way set associative
TLB	32 entries, fully associative
Page size	4 KB

Parameter	Value
External data bus	32 bits
External address bus	32 bits
Crossbar port	32-bit data, dedicated channel

2.2 Pipeline Organization

The processor implements a five-stage pipeline whose stages we name *IF*, *ID*, *EX*, *MEM*, and *WB*.

IF — Instruction Fetch. The contents of the program counter are applied to the instruction cache and to the instruction-side TLB simultaneously. On a hit, a 32-bit instruction is returned at the end of the cycle and latched. The next-PC predictor unconditionally selects PC+4; conditional branches are resolved in EX, and a misprediction is paid for by squashing the instruction in ID. Reset, exception, and unconditional branch redirect the next-PC at the end of the cycle in which they are recognized.

ID — Decode and Register Read. The latched instruction is decoded; the source registers (general or object, as required) are read from their respective register files; the immediate is sign- or zero- extended; hazards against EX and MEM are detected and the appropriate forwarding paths are armed. Reading an object register against a descriptor-cache lookup begins in this stage and completes in EX.

EX — Execute. The arithmetic-logic unit produces its result. For loads and stores through general registers, the effective address is computed. For loads and stores through object registers, the effective offset is computed in parallel with the descriptor cache lookup; the validity, bounds, and capability checks complete by the end of EX, and a failure raises a precise trap that suppresses the remaining stages of the instruction. For branches, the comparison is performed and the next-PC is redirected on the EX-stage clock edge.

MEM — Memory Access. For loads and stores through general registers, the data cache is consulted; on a hit, the data are returned at the end of the cycle. For loads and stores through object registers, either the data cache (for local objects) or the crossbar interface (for remote objects) is engaged. A cache miss stalls the pipeline; a remote access stalls until the response packet returns. Stores complete in this stage; their data are committed to the cache on the cycle's clock edge and to the write buffer for propagation to main memory.

WB — Writeback. The result of the instruction, if any, is written to the destination register on the rising edge of the WB-stage clock. The instruction is retired.

2.3 Hazards and Forwarding

Two forwarding paths short-circuit the register file: from the EX- stage ALU output to the EX-stage ALU input, and from the MEM-stage load result to the EX-stage ALU input. The first eliminates the read-after-write delay on chains of arithmetic instructions; the second reduces, but does not eliminate, the cost of a load-use sequence.

Load-use hazards are *not* hardware-interlocked, in keeping with the architectural specification of Volume II. The compiler is expected to schedule an unrelated instruction into the load-delay slot. If it cannot, a `NOP` is inserted; the result of the load is undefined if read in the immediately succeeding instruction. The compiler-fill rate observed in the reference toolchain on representative C workloads is approximately 70%.

Branch delay slots are likewise compiler-filled; the observed fill rate is approximately 60%.

The multiplier and divider, described in Section 2.8, run independently of the main pipeline. A consumer of `HI` or `LO` that issues before the producer has completed stalls in MEM until the producer retires its result.

2.4 Instruction Cache

The instruction cache is 4 KB, direct-mapped, with 16-byte lines and 20-bit tags. Eight bits of the address index the 256 lines; four bits select the byte within the line. On a miss, the cache requests a 4-word burst from main memory; the missed instruction is forwarded from the bus to the IF stage as it arrives, and the remaining three words are written into the cache concurrently. The fill takes seven cycles in the reference memory configuration.

The cache is virtually indexed and physically tagged. The instruction-side TLB is consulted in parallel with the cache lookup; the physical tag from the TLB is compared with the cache tag at the end of the IF cycle. Misalignment of the program counter raises `address-misaligned-i` before the lookup completes.

Cache lines are not snooped from the data cache; self-modifying code must invalidate the affected line through a privileged firmware primitive before the modified instruction is fetched.

2.5 Data Cache

The data cache is also 4 KB, direct-mapped, with 16-byte lines. It is write-through and no-write-allocate: stores update the cache on a hit but do not allocate on a miss; in either case the store is forwarded to a four-entry write buffer that drains to main memory asynchronously. The buffer permits up to four pending writes to be absorbed before stores stall the pipeline.

The data cache is virtually indexed and physically tagged in the same manner as the instruction cache. A cache miss services a 4-word burst, taking seven cycles in the reference memory configuration; the requested word is forwarded to MEM as it arrives.

The cache is not coherent with the crossbar: remote accesses bypass the local data cache entirely, dispatching directly to the crossbar interface. This is the practical consequence of the architectural decision recorded in Volume I, Section 7, and avoids the substantial complexity that any cross-crossbar coherence protocol would impose.

2.6 Object Descriptor Cache

The object descriptor cache (ODC) is 32 entries, organized as eight sets of four ways. The tag is the 32-bit concatenation of the home processor identifier and the local table index of the cached

object (8 + 24 bits, with the upper 8 bits zero in single-crossbar configurations). The data portion of each entry is a 32-byte descriptor; the cache therefore occupies 1 KB of descriptor storage plus tag and replacement state.

Lookup is performed in parallel with effective-address computation during EX. A hit returns the descriptor in time for the bounds and capability checks to complete in the same cycle; a miss stalls the pipeline and dispatches a `DESC_REQ` packet to the home processor through the crossbar interface. The reference implementation observes ODC miss rates below one percent on representative workloads of the firmware reference test suite.

Replacement within a set is least-recently-used. Firmware may flush the ODC explicitly by writing the appropriate firmware-only control register (Section 2.10).

2.7 Translation Lookaside Buffer

The TLB is 32 entries, fully associative. Each entry holds a 20-bit virtual page number, an 8-bit address-space identifier, a 20-bit physical page number, and a 4-bit permission field encoding read, write, execute, and global bits. On every memory access, the TLB is queried; a miss raises `tlb-miss-i` or `tlb-miss-d` to firmware, which is responsible for loading the appropriate entry through the `TLBWI` and `TLBWR` instructions.

The address-space identifier permits multiple per-task address spaces to coexist in the TLB without flushing on context switch; firmware loads the current task's ASID into the appropriate field of the context register before resuming the task.

Replacement under `TLBWR` is governed by the value of the `RANDOM` control register, which decrements on every cycle and wraps modulo the TLB size; firmware that wishes to pin a small set of entries may do so by maintaining `RANDOM` above the pinned region's index.

2.8 Multiplier and Divider

The multiplier is a 16×16 booth array iterated twice per 32×32 operation; a `MULT` or `MULTU` completes in twelve cycles and deposits the 64-bit product in `HI:LO`. The divider is a non- restoring serial unit producing one bit per cycle; a `DIV` or `DIVU` completes in thirty-five cycles. Both units run concurrently with the main pipeline; only a consumer of `HI` or `LO` stalls.

The multiplier and divider are unprotected from sign issues at the boundary cases (`INT_MIN / -1`); firmware traps `arithmetic-overflow` on the offending result.

2.9 External Bus and Crossbar Interface

The OR-1000 presents two distinct external interfaces: a conventional synchronous memory bus carrying 32 bits of data and 32 bits of address with associated control, and a dedicated crossbar port carrying 32 bits of data and the small handful of strobes required by the protocol of Volume IV.

The memory bus is multiplexed in the conventional manner, with bus transactions taking five cycles in the reference configuration: one to drive the address, three to transfer data (one per word of a four-word burst is the typical case), and one for turnaround.

The crossbar port operates synchronously to the same clock as the processor and carries one packet word per cycle in each direction. The interface buffers up to four packets in each direction; flow-control credits are managed in hardware according to Volume IV, Section 7.

2.10 Control Registers

The architecturally visible control registers, accessed through `LCTRL` and `SCTRL`, are numbered as follows. The accessibility column indicates whether supervisor mode (sv) or only firmware (fw) may read or write each register.

Number	Name	Access	Contents
0	`STATUS`	sv/fw	Current mode, interrupt mask
1	`CAUSE`	sv/fw	Last exception cause and BD bit
2	`EPC`	sv/fw	Exception program counter
3	`BADVADDR`	sv/fw	Faulting virtual address
4	`CONTEXT`	sv/fw	Page-table base and current ASID
5	`COUNT`	sv/fw	Free-running cycle counter
6	`COMPARE`	sv/fw	Timer interrupt compare value
7	`PROCID`	sv/fw	This processor's identifier (read-only)
8	`VECBASE`	fw	Base of the firmware vector table
9	`TLBHI`	fw	TLB entry: virtual page number, ASID
10	`TLBLO`	fw	TLB entry: physical page, permissions
11	`INDEX`	fw	Index for `TLBWI`, `TLBR`
12	`RANDOM`	fw	Index for `TLBWR` (decrements freely)
13	`OBJTAB_BASE`	fw	Physical base of object table
14	`OBJTAB_LIMIT`	fw	Number of slots in object table
15	`ROUTE_BASE`	fw	Physical base of routing table
16	`ODC_TAG`	fw	Tag of ODC entry being read or written
17	`ODC_DATA`	fw	Data of ODC entry being read or written
18–31	reserved		Read as zero; writes ignored

The supervisor-accessible subset (registers 0–7) suffices for the trap-handling and cycle-accounting needs of an operating system; the firmware-only subset (registers 8 and above) controls the TLB, the object table, the crossbar routing, and the descriptor cache.

3. The OR-XBAR-1 Crossbar

3.1 Specifications

Parameter	Value
Process	1.5-micron, two-layer-metal CMOS
Transistors	approximately 80,000
Die size	8.5 mm × 8.5 mm
Pin count	264 (PGA)
Clock	16 MHz nominal
Ports	16, each 32-bit data plus control
Input FIFO depth	32 words per port
Arbitration latency	1 cycle per output port
Switch traversal	1 cycle

3.2 Port Organization

Each of the sixteen ports presents an identical electrical interface: 32 bits of data in each direction, four bits of control encoding packet-word framing and link-control, and the shared synchronous clock. The transmit and receive channels are independent and may be driven in the same cycle.

A port's input FIFO is 32 words deep, sized as Volume IV Section 7 requires. The output side has no FIFO of its own; arbitration grants the use of the switching matrix for one packet at a time, with the source's output stream latched directly into the matrix on grant.

3.3 Arbitration

Arbitration at each output port is rotating-priority, as Volume IV Section 5 specifies. The implementation is a 16-bit one-hot state register holding the current priority pointer, combined with a 15-input request mask gated by the pointer; the granted input is selected by the low-order set bit of the masked request vector. The implementation completes in approximately 8 nanoseconds in 1.5-micron CMOS, comfortably within a single 50-nanosecond cycle at 20 MHz.

3.4 Switching Matrix

The matrix is a 16×16 fully-connected crosspoint of 32-bit lanes, implemented as 32 independent 16-input multiplexers, one per data bit. The control inputs of all 32 multiplexers in a row are tied together and driven by the granted-input signal from the corresponding output port's arbiter.

3.5 Diagnostic and Reset

A single diagnostic port, electrically distinct from the sixteen data ports, presents a small register file accessible by an attached service processor. The registers permit read-out of per-port packet counts, error counts, and the current priority pointer of each output port, and allow firmware-controlled assertion of `LINK_RESET` on any subset of ports.

4. System Configurations

4.1 Single-Processor Workstation

The minimum conforming Object RISC configuration consists of one OR-1000, four to sixteen megabytes of dynamic memory, a small ROM holding the firmware image, and an I/O subsystem of the integrator's choosing. The crossbar is omitted; the processor's crossbar port is either left unconnected or, in some implementations, looped back through a small on-board controller that responds to messages addressed to processor identifiers other than zero with a `MISDIRECTED` link error.

The single-processor configuration exercises the entire architecture except for inter-processor communication. The firmware reduces to a minimal kernel of task management, memory management, and I/O; the hypervisor primitives of Volume VI degrade gracefully into a single-tenant model.

4.2 Four-Way and Eight-Way Server

The intermediate configurations comprise four or eight OR-1000 processors connected to a single OR-XBAR-1 with the unused ports unconnected. Each processor has its own local memory, of typically sixteen megabytes; total system memory is therefore 64 to 128 megabytes, partitioned among the processors and reachable globally through the crossbar.

The unused ports of the crossbar are a sunk cost in silicon; the chip is not made smaller for smaller configurations. Integrators seeking a lower-cost intermediate scaling may instead use a smaller crossbar, of which two- and four-port variants have been designed internally and are described in supplementary documents not part of the present revision.

4.3 Sixteen-Way Server

The maximum single-crossbar configuration uses all sixteen ports of one OR-XBAR-1, connecting sixteen OR-1000 processors with their local memory. A typical 1986 cabinet houses two such configurations (a "two-cluster" system of thirty-two processors) with the clusters joined by a hierarchical second-level crossbar; processor identifiers 0 through 15 are assigned to one cluster and 16 through 31 to the other.

The hierarchical configuration is not a tested reference in this revision but is supported by the protocol of Volume IV. Volume V concerns itself only with the contents of a single cluster.

4.4 Reset and Boot Sequence

On power-up or reset, every OR-1000 enters firmware mode with all interrupts masked and the program counter set to a fixed reset vector address. The reset vector is at offset zero of the firmware ROM, which is mapped at a physical address determined by the configuration of two strap pins.

The reset sequence proceeds as follows:

1. Each processor reads its `PROCID` and selects the appropriate branch of the boot code: processor zero is the *boot processor* and proceeds with system initialization; processors of higher

identifier execute a small wait loop, polling a memory location that the boot processor will later write.

2. The boot processor sizes its local memory by a destructive write- read sweep, initializes the object table at a firmware-chosen physical base, programs `OBJTAB_BASE` and `OBJTAB_LIMIT`, clears the TLB and ODC, and configures `VECBASE`.
3. The boot processor configures the crossbar by writing routing tables (in hierarchical configurations) through the diagnostic port, and exchanges `LINK_HEARTBEAT` with each peer to confirm connectivity.
4. The boot processor releases the other processors by writing the well-known location they are polling; each then performs an abbreviated form of steps 2 and 3 against its local resources, omitting the global crossbar configuration.
5. Each processor proceeds to load the operating system image from the boot device, transferring control out of firmware by `ERET`.

The reset sequence is timed at approximately 50 milliseconds in the reference implementation, dominated by the memory-sizing sweep.

5. Physical Characteristics

5.1 Pin Assignment Summary

The OR-1000's 168 pins partition as follows:

Function	Pin count
Memory address bus	32
Memory data bus	32
Memory control	12
Crossbar transmit data	32
Crossbar receive data	32
Crossbar control	8
Interrupts	8
Clock and reset	4
Test, diagnostic, and strap	8
Power and ground	32 (16 pairs)

The OR-XBAR-1's 264 pins are dominated by the sixteen ports of 36 pins each (32 data + 4 control bidirectional), with the remainder dedicated to power, clock, and the diagnostic interface.

5.2 Power and Thermal

The OR-1000 dissipates 2.4 W typical at 16 MHz and 5 V; selected parts at 20 MHz dissipate 3.0 W. The OR-XBAR-1 dissipates 1.8 W typical at 16 MHz. Both chips are specified for operation at junction temperatures up to 85°C and require no forced air in representative cabinets.

6. Performance

6.1 Single-Processor Workloads

Measured cycles per instruction on the firmware reference test suite:

Workload	CPI	Notes
Integer arithmetic kernel	1.05	Compiler-scheduled
Pointer-traversal kernel	1.40	Cache fits working set
Object-register-heavy kernel	1.18	All ODC hits
Operating-system mix	1.55	Includes TLB-miss handling

At 20 MHz the corresponding native performance falls in a range broadly comparable to first-generation MIPS R2000 and SPARC parts of the same era; the architecture's distinguishing features impose no measurable penalty on integer code that does not exercise them.

6.2 Inter-Processor Communication

Measured latencies on a 16-processor reference configuration at 16 MHz:

Operation	Cycles	Time
Local `OL*` (descriptor cache hit)	2	125 ns
Remote `OL*` (descriptor cache hit, no contention)	28	1.75 µs
Remote `OL*` (descriptor cache miss)	58	3.6 µs
`SEND` issue to handler entry	22	1.4 µs
`CALL` to a leaf firmware primitive	18	1.1 µs

These numbers compare favourably with the message-passing latencies of contemporary tightly-coupled multiprocessors, and we believe they validate the decision to make the crossbar a primitive of the architecture rather than a peripheral.

6.3 Comparative Position

Among contemporary architectures of similar scale and process, the OR-1000 is most directly comparable to the MIPS R2000 in single- processor performance and to the INMOS T800 transputer in communication latency. We are unaware of any commercially available architecture in 1986 that combines the two within a single instruction set, and we offer this combination as the principal contribution of the work.

Volume VI — System Firmware Interface

1. Scope

This volume specifies the system primitive interface — the set of operations reached from user-mode and supervisor-mode code through the `CALL` instruction. It defines the calling convention, the allocation of primitive numbers among functional groups, the arguments, return values, and error conditions of each primitive in this revision, and the conformance requirements that any firmware implementation claiming to be Object RISC firmware must meet.

The architecture-level rationale for the existence of System Firmware as a layer is given in Volume I, Section 6. The trap mechanism by which `CALL` is dispatched, and the layout of the firmware vector table, are specified in Volume II. This volume specifies the behaviour above the trap.

The reader is assumed to be familiar with all preceding volumes.

2. The Primitive Call Convention

2.1 Argument and Return Registers

Arguments to a primitive are passed in `R4`–`R7` and `01`–`04`, in the order in which they appear in the primitive's signature: integer arguments fill `R4`, `R5`, `R6`, `R7` in order, and reference arguments fill `01`, `02`, `03`, `04` in order. A primitive of fewer than four integer or four reference arguments leaves the unused registers undefined; the caller is responsible for populating only the registers the primitive consumes.

Return values are returned in `R2` and `R3` (integer) and `01` (reference). A primitive that returns no value leaves these registers unmodified from the firmware's perspective; the caller may not assume they retain their pre-call value.

The primitive number is encoded in the 26-bit immediate of the `CALL` instruction itself, as Volume II specifies. It is not passed in a register.

2.2 Caller- and Callee-Saved State

Firmware preserves all registers visible to the caller across a `CALL` except for `R2`, `R3`, `01`, and any register the caller has explicitly populated as an argument. The standard caller-saved temporaries (`R8`–`R15`, `R24`–`R28`, `05`–`08`, `013`–`015`) are saved by firmware on entry and restored on return. This convention is more conservative than the calling convention of Volume II requires, and is justified

by the desire that `CALL` behave indistinguishably from a procedure call for purposes of compiler register allocation.

2.3 Error Reporting

Every primitive returns a status code in `R2`. Zero indicates success; a nonzero value identifies an error condition per Section 2.4. A primitive that returns useful values in `R3` or `01` does so only on success; on error those registers carry undefined contents.

A primitive that requires a capability bit on a reference argument returns `EPERM` if the capability is absent, regardless of whether other invariants on the argument hold. A primitive that requires a live reference and is given a stale or null one returns `ESTALE` or `EFAULT` respectively, before any capability check is performed.

2.4 Standard Error Codes

Code	Symbol	Meaning
0	`OK`	Success
1	`EINVAL`	Invalid argument value
2	`ENOMEM`	Insufficient memory or table space
3	`EPERM`	Required capability bit absent on a reference
4	`ENOSYS`	Primitive not implemented in this firmware
5	`EBUSY`	Resource is in use and cannot be acquired now
6	`ENOENT`	No such object, device, or named resource
7	`ETIMEDOUT`	Bounded wait expired before completion
8	`EFAULT`	Reference is null where prohibited
9	`EAGAIN`	Transient failure; the caller should retry
10	`ESTALE`	Reference's generation does not match the descriptor
11	`EREMOTE`	Operation requires a local object; argument is remote

Error codes 12 through 255 are reserved for future revisions. Implementations may not use them for implementation-specific errors; such errors are reported through `EINVAL` with a supplementary diagnostic written to the firmware log.

2.5 Restartability

The architecture distinguishes *restartable* primitives, which firmware may abandon partway and which the caller may safely re-issue without duplicating any side effect, from *non-restartable* primitives, which once entered run to completion. Restartability is documented per primitive in the descriptions that follow. A primitive interrupted by an external event (e.g., the expiration of a scheduling quantum) is either resumed at the point of interruption or restarted from arguments; the architecture permits either choice and requires only that the choice be transparent to the caller.

3. Primitive Number Allocation

Primitive numbers are partitioned into functional groups:

Range	Group
`0x000–0x0FF`	Task management
`0x100–0x1FF`	Memory management
`0x200–0x2FF`	Communication
`0x300–0x3FF`	Device and I/O
`0x400–0x4FF`	Time and clocks
`0x500–0x5FF`	Hypervisor and system management
`0x700–0x7FF`	Diagnostic
All others	Reserved for future extension

A primitive number outside any allocated range raises the `reserved-call` trap as Volume II specifies; firmware terminates the calling task or returns `ENOSYS` per its policy.

4. Task Management Primitives

4.1 Task Lifecycle

`0x000` `TaskCreate` — *Restartable.*

Create a new task and prepare it to run. The task does not run until a subsequent `TaskResume`.

Args: - `O1`: code object (must carry `X`). - `O2`: initial stack object (must carry `R` and `W`). - `R4`: byte offset within the code object at which to begin. - `R5`: initial value to place in `R4` of the new task.

Returns: - `O1`: reference to the created task object, with `V` capability. - `R2`: status.

Errors: `EPERM`, `ENOMEM`, `EFAULT`, `EINVAL`.

`0x001` `TaskExit` — *Non-restartable.*

Terminate the calling task with the given exit code. The task object may be queried for its exit code via `TaskQuery` after termination.

Args: - `R4`: exit code.

Does not return.

`0x002` `TaskResume` — *Restartable.*

Place the named task in the scheduler's runnable set.

Args: - `O1`: task object (must carry `V`).

Returns: R2: status. Errors: EPERM, ESTALE, EBUSY.

0x003 TaskSuspend — *Restartable.*

Remove the named task from the scheduler's runnable set. A task may suspend itself; doing so blocks until another task resumes it.

Args: - O1: task object (must carry V).

Returns: R2: status. Errors: EPERM, ESTALE.

0x004 TaskYield — *Restartable.*

Surrender the remainder of the current scheduling quantum. The task remains runnable.

No arguments. Returns: R2: status (always OK).

0x005 TaskCurrent — *Restartable.*

Return a reference to the calling task's task object.

No arguments. Returns: O1: calling task's reference (caps include V).

0x006 TaskBindProcessor — *Restartable.*

Suggest a processor on which the task should preferentially run. The firmware is not required to honour the suggestion.

Args: - O1: task object (must carry V). - R4: preferred processor identifier, or oxFF for "no preference".

Returns: R2: status.

0x007 TaskWait — *Non-restartable.*

Block the calling task until the named task has terminated. The exit code of the awaited task is returned in R3.

Args: - O1: task object (no capability required).

Returns: R2: status. R3: exit code on success.

0x008 TaskQuery — *Restartable.*

Return summary information about a task: its state (runnable, suspended, terminated), its current processor, and its exit code if terminated.

Args: - O1: task object.

Returns: R2: status. R3: packed state word (state in low 8 bits, processor in next 8, exit code in upper 16).

4.2 Synchronization

`0x010` `SemAlloc` — *Restartable.*

Allocate a counting semaphore with the given initial count. Returns a reference to the semaphore as an object; capabilities are determined by the policy of the caller's address space.

Args: - `R4`: initial count.

Returns: `O1`: semaphore reference. `R2`: status.

`0x011` `SemAcquire` — *Non-restartable.*

Decrement the semaphore, blocking the calling task if the count is zero. The blocking is bounded by an optional timeout.

Args: - `O1`: semaphore reference (must carry `S`). - `R4`: timeout in clock ticks, or zero for no wait, or oxFFFFFFFF for unbounded wait.

Returns: `R2`: status. Errors include `ETIMEDOUT`.

`0x012` `SemRelease` — *Restartable.*

Increment the semaphore, possibly waking a blocked acquirer.

Args: - `O1`: semaphore reference (must carry `S`).

Returns: `R2`: status.

`0x020` `EventAlloc`, `0x021` `EventWait`, `0x022` `EventNotify`

A binary event is a semaphore with the constraint that its count never exceeds one. The three primitives mirror the semaphore primitives in their argument and return conventions.

5. Memory Management Primitives

5.1 Object Lifecycle

`0x100` `ObjAlloc` — *Restartable.*

Allocate a new object on the calling processor.

Args: - `R4`: requested length in bytes. - `R5`: type tag (low 16 bits significant). - `R6`: initial maximum capabilities (low 8 bits significant).

Returns: - `O1`: reference to the new object, with capabilities equal to the requested maximum. - `R2`: status.

Errors: `ENOMEM`, `EINVAL` (length zero or larger than implementation maximum).

`0x101` `ObjFree` — *Non-restartable.*

Increment the descriptor's generation counter and return its storage to the local free pool. All outstanding references to the object become stale.

Args: - `O1`: object reference (must carry `V`).

Returns: `R2`: status. Errors: `EPERM`, `EREMOTE`.

`0x102` `ObjRevoke` — *Non-restartable.*

Increment the descriptor's generation counter without freeing the storage. The object's slot remains allocated and may be re-bound by firmware to a fresh storage region in a subsequent operation.

Args: - `O1`: object reference (must carry `V`).

Returns: `R2`: status. Errors: `EPERM`, `EREMOTE`.

`0x103` `ObjDerive` — *Restartable.*

Produce a new reference to the same object, with capabilities equal to the bitwise AND of the input reference's capabilities and the supplied mask.

Args: - `O1`: reference (must carry `C`). - `R4`: capability mask (low 8 bits significant).

Returns: `O1`: derived reference. `R2`: status.

`0x104` `ObjMigrate` — *Non-restartable.*

Move the named object to the named processor's local memory, leaving a forwarding stub at the original home as Volume III, Section 6.4 describes.

Args: - `O1`: object reference (must carry `M`). - `R4`: destination processor identifier.

Returns: `R2`: status. Errors: `EPERM`, `EBUSY` (migration already in progress), `ENOMEM` (destination cannot allocate slot or storage).

`0x105` `ObjQuery` — *Restartable.*

Return summary information about an object: its current home, its length, its type tag, and its maximum capabilities. The reference's own effective capabilities are not returned (the caller already holds them).

Args: - `O1`: reference.

Returns: - `R2`: status. - `R3`: packed (home in low 8, max_caps in next 8, type_tag in upper 16). - On success a second word is delivered through the implementation's side-channel — see Section 11.

5.2 Address-Space Mapping

`0x110` `MapObject` — *Restartable.*

Install page-table entries that resolve a chosen virtual range to the storage of the named object. The object's home must be the calling processor.

Args: - `O1`: object reference. The bits selected by `R6` of this reference's capabilities must include those mode bits requested. - `R4`: virtual address hint, or zero for "any". - `R5`: byte offset within the object at which the mapping begins. - `R6`: protection bits requested (low 3 bits: R, W, X). - `R7`: length of the mapping in bytes.

Returns: `R2`: status. `R3`: virtual address at which the mapping was installed.

Errors: `EREMOTE`, `EPERM`, `EINVAL`, `ENOMEM`.

`0x111` `Unmap` — *Restartable.*

Remove a mapping previously established by `MapObject`. The object's identity is recovered from the page table; no reference is required.

Args: - `R4`: virtual address. - `R5`: length.

Returns: `R2`: status.

`0x112` `Protect` — *Restartable.*

Modify the protection bits of an existing mapping. The new protection must be a subset of the capabilities the underlying reference originally carried at `MapObject` time; this constraint is enforced by firmware against the per-mapping record.

Args: - `R4`: virtual address. - `R5`: length. - `R6`: new protection bits.

Returns: `R2`: status.

5.3 Object Register Spill

The architecture provides no instruction by which an object register's contents may be stored to or loaded from a general-register address, on the grounds given in Volume III: to permit it would defeat the capability invariant. The firmware therefore mediates the only permitted form of object-register spill, into a per-task private buffer that is not visible to user-mode integer access.

`0x121` `ORegSpill` — *Restartable.*

Save the contents of a named object register to a slot in the calling task's private spill buffer. Slots are addressed by index within the buffer; sixteen slots are available in any conforming firmware.

Args: - `R4`: object register number to spill (0–15). - `R5`: spill buffer slot index (0–15).

Returns: `R2`: status. Errors: `EINVAL`.

`0x122` `ORegRestore` — *Restartable.*

Restore an object register from a previously-spilled slot. The contents of the slot are unaffected; subsequent restores from the same slot return the same reference.

Args: - `R4`: object register number to restore (0–15). - `R5`: spill buffer slot index (0–15).

Returns: `R2`: status. Errors: `EINVAL`, `ENOENT` (slot empty).

The spill buffer is sized at sixteen slots in this revision. Code that requires deeper spilling must marshal references through a heap-allocated array object as Volume VII Section 2.4 describes.

6. Communication Primitives

`0x200` `InstallHandler` — *Restartable.*

Install a send handler on the named target object. The handler is identified by a code object and a byte offset.

Args: - `O1`: target object (must carry `S` and `V`). - `O2`: handler code object (must carry `X`). - `R4`: byte offset within the handler code object.

Returns: `R2`: status.

`0x201` `RemoveHandler` — *Restartable.*

Remove the send handler from the named target object. Subsequent `SEND`s to the object are dropped per the policy of Volume III, Section 7.2.

Args: - `O1`: target object (must carry `V`).

Returns: `R2`: status.

`0x202` `SendBuffered` — *Restartable.*

Issue a `SEND` whose target is known to be at risk of crossbar back-pressure, queueing the message in firmware-managed memory if the immediate `SEND` would stall or trap. The arguments mirror those of the `SEND` instruction.

Args: - `O1`: recipient object (must carry `S`). - `O2`–`O4`: object payload. - `R4`–`R7`: integer payload.

Returns: `R2`: status. Errors: `EAGAIN` (firmware queue is full).

`0x203` `ReceiveQueueAttach` — *Restartable.*

Replace the dispatch behaviour of the named target object: instead of invoking a handler, the firmware enqueues incoming messages on a per-object queue from which the calling task may explicitly pull.

Args: - `O1`: target object (must carry `S` and `V`). - `R4`: maximum queue depth.

Returns: `R2`: status.

`0x204` `ReceiveQueuePoll` — *Non-restartable.*

Dequeue the next message from a previously attached receive queue, blocking up to a timeout if the queue is empty.

Args: - `O1`: target object (must carry `V`). - `R4`: timeout in clock ticks.

Returns: - R2: status (ETIMEDOUT if no message arrived). - O1–O4: object payload of the dequeued message. - R3–R6: integer payload (note R3 carries the first integer argument here, not a return value, by exception to the usual convention).

0x205 MessagePayloadOR4 — *Restartable.*

Return the fourth object reference from the SEND payload that dispatched the calling handler task. Volume III Section 7.2 specifies that handler dispatch overrides O1 with a self-reference and delivers the wire's first three object references in O2–O4; the fourth reference is held in the side-channel buffer (Section 11) and retrieved through this primitive.

Valid only when called from within a handler task whose dispatch originated as a SEND_DELIVER packet.

No arguments.

Returns: O1: the fourth payload reference (or null if the SEND's fourth payload slot was null). R2: status. Errors: EINVAL (not in a handler dispatch context).

0x210 ProcessorList — *Restartable.*

Populate a buffer with the identifiers of every processor in the system reachable through the crossbar.

Args: - O1: buffer object (must carry W). - R4: byte offset at which to begin writing. - R5: maximum number of identifiers to write.

Returns: R2: status. R3: number of identifiers actually written.

7. Device and I/O Primitives

The set of devices present on a particular system is determined by the integrator and discovered at runtime through the primitives of this section. The architecture does not enumerate device classes; an operating system identifies devices by the type tags of the device objects returned from DeviceList, with conventions established by the firmware integrator and documented in supplementary materials.

0x300 DeviceList — *Restartable.*

Populate a buffer with references to every device object currently registered with firmware.

Args: - O1: buffer object (must carry W). - R4: byte offset. - R5: maximum number of references to write.

Returns: R2: status. R3: number of references actually written.

0x301 DeviceQuery — *Restartable.*

Return identifying information about a device: its class tag, its revision, and a small fixed-format identifier string.

Args: - `O1`: device reference. - `O2`: caller-supplied buffer for identifier string (must carry `W`).

Returns: `R2`: status. `R3`: packed (class in low 16, revision in upper 16).

`0x310` `InterruptInstall` — *Restartable.*

Install a handler for the interrupt source named by the device object. The handler is invoked in the same manner as a `SEND` handler when the interrupt fires.

Args: - `O1`: device reference (must carry `V`). - `O2`: handler code object (must carry `X`). - `R4`: handler offset.

Returns: `R2`: status.

`0x311` `InterruptMask` and **`0x312` `InterruptUnmask`** — *Restartable.*

Disable or enable the interrupt source. Both take a device reference bearing `V` and return a status. Repeated mask operations are idempotent.

`0x320` `ConsoleWrite` and **`0x321` `ConsoleRead`** — *Restartable on read-side timeout, otherwise non-restartable.*

Read or write the firmware-provided console. The console is the minimum I/O capability that any conforming firmware must expose; its precise device class is implementation-defined.

`ConsoleWrite` arguments: - `O1`: source buffer (must carry `R`). - `R4`: byte offset. - `R5`: byte count.

`ConsoleRead` arguments mirror these with `W` on the destination.

Both return `R2`: status, `R3`: bytes actually transferred.

8. Time and Clock Primitives

`0x400` `TimeNow` — *Restartable.*

Return the current value of the system time, expressed as a 64-bit count of clock ticks since boot.

No arguments. Returns: `R2`: status (always `OK`). `R3`: low 32 bits. The high 32 bits are returned through the side-channel of Section 11.

`0x401` `TimerCreate` — *Restartable.*

Schedule the dispatch of a handler at a specified absolute system time.

Args: - `O1`: handler object (must carry `X`). - `R4`: handler offset. - `R5`: low 32 bits of the absolute deadline. - `R6`: high 32 bits of the absolute deadline.

Returns: `O1`: timer object (carries `V`). `R2`: status.

`0x402` `TimerCancel` — *Restartable.*

Cancel a previously scheduled timer. Cancellation is best-effort: a handler whose dispatch has already begun is permitted to run to completion.

Args: - `O1`: timer object (must carry `V`).

Returns: `R2`: status.

`0x403` `SleepUntil` — *Non-restartable.*

Block the calling task until the system time reaches the named absolute deadline.

Args: - `R4`: low 32 bits of deadline. - `R5`: high 32 bits.

Returns: `R2`: status.

`0x410` `ClockResolution` — *Restartable.*

Return the number of clock ticks per second.

No arguments. Returns: `R2`: status. `R3`: ticks per second.

9. Hypervisor and System Management Primitives

The hypervisor primitives are optional. Firmware that does not implement guest hosting returns `ENOSYS` from each of them; the presence or absence of hypervisor support is reported by the diagnostic primitive `Stat` (Section 11).

`0x500` `GuestCreate` — *Restartable.*

Allocate a guest container, into which a guest operating system image may subsequently be loaded.

Args: - `R4`: maximum memory in bytes the guest may consume. - `R5`: bitmap of processors permitted to run guest tasks (low 16 bits).

Returns: `O1`: guest reference (carries `V`). `R2`: status.

`0x501` `GuestLoad` — *Restartable.*

Load an executable image into the guest's address space. The guest must be in the unstarted state.

Args: - `O1`: guest reference (must carry `V`). - `O2`: image object (must carry `R`).

Returns: `R2`: status.

`0x502` `GuestStart` — *Non-restartable.*

Begin execution of the guest at its image's entry point.

Args: - `O1`: guest reference (must carry `V`).

Returns: `R2`: status.

`0x503` `GuestStop` — *Non-restartable.*

Halt the guest. The guest's state is preserved and may be inspected through `GuestQuery`.

Args: - `O1`: guest reference (must carry `V`).

Returns: `R2`: status.

`0x504` `GuestQuery` — *Restartable.*

Return summary information about a guest: its state, its memory consumption, and the number of tasks it has created.

Args: - `O1`: guest reference.

Returns: `R2`: status. `R3`: packed (state in low 8 bits, processor count in next 8, task count in upper 16).

`0x510` `SystemReset` — *Non-restartable.*

Reset the local processor as if by hardware reset. Other processors are unaffected.

No arguments. Does not return.

`0x511` `SystemHalt` — *Non-restartable.*

Halt the local processor pending external intervention. Other processors are unaffected.

No arguments. Does not return.

10. Diagnostic Primitives

`0x700` `Stat` — *Restartable.*

Return a counter from the firmware's instrumentation: total primitives invoked, total `SEND`s issued, total page faults serviced, and so on. The set of available counters is implementation-defined; well-known counters in the range 0–63 are listed in supplementary documents.

Args: - `R4`: counter identifier.

Returns: `R2`: status. `R3`: counter low 32 bits. High 32 bits are returned through the side-channel.

`0x701` `ProcessorID` — *Restartable.*

Return the calling processor's identifier and a bitmap of features the firmware reports on it.

No arguments. Returns: `R2`: status. `R3`: packed (identifier in low 8, feature bits in upper 24).

0x702 DebugPrint — *Restartable.*

Append a fixed-format diagnostic record to the firmware log. The primitive is intended for debugging and is permitted to be a no-op in production firmware images.

Args: - O1: source buffer (must carry R). - R4: byte offset. - R5: byte count.

Returns: R2: status.

11. The Side-Channel

Several mechanisms in this revision require carrying more state into or out of a primitive than fits in the architectural argument and return registers. Those mechanisms use a *side-channel*: a small fixed-size object, allocated at task creation, whose reference is held implicitly in the task's control block and whose contents firmware reads or writes during the relevant operation.

The side-channel object is RBUF bytes long, where RBUF is at least 64 in any conforming firmware. The layout of its contents is specified per usage; this revision uses the side-channel in two distinct ways.

Overflow returns. TimeNow, Stat, and ObjQuery deposit return values that do not fit in R2, R3, or O1 into the side-channel, where the caller reads them on return. The high 32 bits of the 64-bit values returned by TimeNow and Stat occupy the first eight bytes of the buffer; ObjQuery writes its second word at offset 16.

Implicit input from dispatch. When firmware dispatches a handler task in response to SEND, the wire-format fourth object reference of the payload (which has no register slot in the dispatch convention of Volume III Section 7.2) is written to the dispatched task's side-channel at offset 32. The handler retrieves it through MessagePayloadOR4 (Section 6).

The side-channel exists to keep the common-case calling convention simple at the cost of an extra firmware-internal indirection in the rare case where the architectural registers do not suffice. We expect future revisions either to widen the convention or to standardize a richer side-channel structure.

12. Conformance Requirements

A firmware implementation conforming to this revision shall implement every primitive in groups 0x000–0x4FF and 0x700–0x701, including the object-register spill primitives 0x121 and 0x122 and the handler side-channel primitive 0x205. The hypervisor group (0x500–0x5FF) is optional; an implementation that does not provide it returns ENOSYS from every primitive in the group and reports the absence through Stat counter 16 ("hypervisor present", returning zero).

The device discovery primitives DeviceList and DeviceQuery are required, but the set of devices they enumerate is not specified by this volume. Every conforming firmware exposes at least the console device on which ConsoleRead and ConsoleWrite operate.

Implementations may extend the primitive set with locally defined operations in any of the reserved ranges. Code using such extensions is not portable to other Object RISC firmware implementations and should be flagged as such by the toolchain.

The reserved primitive numbers within allocated groups (those not named in this volume) shall return `ENOSYS` and shall not be allocated by implementations to local extensions; future revisions of this volume will consume them.

Volume VII — Programming Practice

1. Scope

This volume describes how the architecture defined in Volumes I through VI is *programmed in practice*: the standard application binary interface, the idioms that have emerged from early use of the object system and the inter-processor communication primitives, a worked example demonstrating the mechanics end-to-end, and a small body of notes on compiler output and performance.

It is more discursive than the preceding volumes, in keeping with its subject. Where the formal specifications are silent on matters of convention, this volume records the conventions adopted by the reference toolchain and recommended for portable code. The conventions here are not part of the architectural contract; they are the recommendations of a reference compiler against which user programs are expected to interoperate.

The reader is assumed to be familiar with all six preceding volumes.

2. The Standard ABI

2.1 Register Usage

The conventions of Volume II Section 3 are summarized below for ease of reference, with the additional information of what each register's contents may be assumed to be across a procedure call.

Register	Role	Across a call
`R0`	Constant zero	Unchanged
`R1`	Assembler temporary	Undefined
`R2`–`R3`	Integer return values	Set by callee
`R4`–`R7`	Integer arguments	Undefined after call
`R8`–`R15`	Caller-saved temporaries	Undefined after call
`R16`–`R23`	Callee-saved	Preserved
`R24`–`R28`	Caller-saved temporaries	Undefined after call
`R29` (`SP`)	Stack pointer	Preserved
`R30` (`FP`)	Frame pointer	Preserved
`R31` (`RA`)	Link register	Set by `JAL`/`JALR`
`00`	Constant null	Unchanged

Register	Role	Across a call
01–04	Object arguments / first OR return	Undefined after call
05–08	Caller-saved temporaries	Undefined after call
09–012	Callee-preserved	Preserved (see Section 2.4)
013–015	Caller-saved temporaries	Undefined after call

The frame pointer is optional. Functions whose stack frame is of constant size and whose alloca-style allocations are absent omit it and use SP as the sole frame anchor; functions with variable-size frames maintain FP and address locals through it.

2.2 Stack Frame Layout

The stack grows toward lower addresses. A function's frame, allocated in its prologue, contains the following regions in descending memory order:

```
+---------------------------+  ← incoming SP
|   caller's argument spill  |
|   16 bytes                 |
+---------------------------+
|   saved RA                 |  4 bytes
|   saved FP (if used)       |  4 bytes
|   saved callee-preserved   |  up to 32 bytes
|   GPRs (R16–R23)           |
+---------------------------+
|   local variables          |
|   (compiler-managed)       |
+---------------------------+
|   outgoing argument spill  |
|   16 bytes                 |
+---------------------------+  ← new SP
```

The 16-byte spill area at the top of every frame is reserved by the caller for the callee to spill its incoming integer arguments (R4–R7) to memory if it needs their addresses, or if it must spill them across an inner call. The matching area at the bottom is reserved by the current function for the next inner call in the same way.

The stack pointer is required to be 8-byte aligned at all call boundaries. Frame sizes are accordingly rounded up to a multiple of 8.

2.3 Procedure Prologue and Epilogue

A canonical prologue for a function with a frame size F that uses the link register and two callee-preserved GPRs R16 and R17:

```
funcname:
```

```
addiu sp, sp, -F          ; allocate frame
sw    r31, F-4(sp)        ; save link register
sw    r17, F-8(sp)        ; save callee-preserved
sw    r16, F-12(sp)
; ... body ...
```

The matching epilogue:

```
lw    r17, F-8(sp)        ; restore callee-preserved
lw    r16, F-12(sp)
lw    r31, F-4(sp)        ; restore link register
jr    r31                 ; return
addiu sp, sp, F           ; (delay slot) deallocate frame
```

The deallocation of the frame in the branch delay slot is a small but useful idiom: the return address has already been loaded into a register and the stack will not be referenced after the jump, so the deallocation is safe to overlap with the return.

A leaf function that does not call others, has no spills, and uses no callee-preserved registers may omit the prologue and epilogue altogether and proceed directly with its body, returning by `jr r31`.

2.4 Object Register Discipline

Object registers are *not spillable to integer memory*. The architecture deliberately prohibits the bit-pattern reconstruction of an object reference from arbitrary memory contents — to do so would defeat the capability invariant Volume III establishes — and provides no instruction by which an object register's contents may be written to or loaded from a general-register address.

The practical consequence is that the four callee-preserved object registers `O9`–`O12` are preserved *by convention*: the callee must not modify them. There is no save-and-restore at the prologue. Code that requires more object registers than the working set provides must, in order of preference:

1. Restructure to use fewer simultaneous references.
2. Marshal references through a heap-allocated array object, storing each reference at a known byte offset and reloading it through `OL`/`OS` when needed (an indirection but a cheap one).
3. Invoke the firmware primitives `ORegSpill` and `ORegRestore` (Volume VI Section 5.3, primitive numbers `0x121` and `0x122`), which save and restore a named object register through the calling task's private firmware-managed spill buffer. The buffer holds sixteen slots in any conforming firmware.

In our experience the conventional working set of fifteen object registers is adequate for substantially all hand-written and compiler- generated code, and the spill primitives are rarely exercised outside of deeply recursive algorithms over object graphs.

2.5 Variadic Arguments

Variadic functions place arguments beyond the first four integer slots in the caller's outgoing argument spill area, ascending from `0(sp)`. The first four arguments occupy `R4`–`R7` as for non-variadic functions; arguments numbered five and beyond occupy `0(sp)`, `4(sp)`, `8(sp)`, ... in order.

Object reference arguments to variadic functions are passed in `O1`–`O4` only; the variadic mechanism does not extend to references, since they cannot be spilled to integer memory. A variadic function that requires more than four reference arguments must accept them through an explicit array-object indirection.

2.6 Returning Aggregates

A struct of total size 8 bytes or less is returned in `R2:R3`, low-word in `R2`. A struct between 9 and 64 bytes is returned through a caller-supplied buffer whose address is passed as an *implicit zeroth argument* in `R4`, with the explicit arguments shifted to `R5`–`R7` and the spill area; the callee writes the struct to the buffer and the caller reads it from the same address. A struct larger than 64 bytes is returned by allocating a fresh object whose reference is returned in `O1`.

A struct that contains object references is always returned through an object reference, by the rule of Section 2.4.

3. Object System Idioms

3.1 Allocate-Initialize-Use

The conventional pattern for allocating an object, initializing its contents, and proceeding to use it:

```
addiu r4, r0, LEN          ; length in bytes
addiu r5, r0, TAG          ; type tag
addiu r6, r0, CAPS         ; initial maximum capabilities
call  #0x100               ; ObjAlloc
bne   r2, r0, alloc_fail
nop                        ; load delay slot for r2

; The new object's reference is in O1 with capabilities equal to
; the requested initial set. Store an initial value into the first
; word of the object:
addiu r3, r0, INITIAL
osw   r3, 0(o1)
```

The `caps` value should be the *least* set of capabilities for which any holder of a reference to this object will ever have a legitimate need. Capabilities cannot be strengthened later; over-allocation weakens the object's protection without recovery. The five most commonly requested combinations are tabulated below.

Caps (hex)	Bits	Typical use
`0x03`	`R W`	Private mutable data (no sharing)
`0x07`	`R W X`	Code object with mutable static data
`0x4F`	`R W S V C`	Service object with revocation and derivation
`0x53`	`R W S V`	Single-recipient service (no derivation)
`0x09`	`S X`	Send-only capability to a fixed handler

3.2 Capability Passing

A reference held with broad capabilities is rarely the reference one should pass to another component. The standard idiom is to derive a weaker reference at the moment of passing:

```
; We hold O9 with caps R|W|S|V|C. We wish to grant a callee the
; ability to read and send to the object, but not to modify or
; revoke it.
omov  o1, o9              ; reference to derive from
addiu r4, r0, 0x09        ; mask = R|S
call  #0x103              ; ObjDerive
bne   r2, r0, derive_fail
nop
omov  o2, o1              ; pass the weakened reference as arg
jal   downstream_function
nop
```

The same technique is used to construct a "reply capability" for asynchronous messaging: the issuer derives a send-only reference (`caps = 0x08`, `S` only) to a private reply buffer and passes it in the `SEND` payload; the recipient may use it only to send back, and specifically may not read or modify the reply buffer's contents.

3.3 Releasing References

There is no garbage collector. References that the program has finished with are dropped by overwriting the holding register, by stack unwinding, or by `ObjFree`/`ObjRevoke` when the object itself is to be released.

A reference whose object outlives the holder need not be released explicitly; overwriting the register suffices. A reference whose object is to be reclaimed, on the other hand, requires `ObjFree` (to release the storage and the table slot) or `ObjRevoke` (to invalidate every outstanding reference without releasing the storage). The conventional discipline is:

- *Locally allocated, locally consumed* objects: the allocator calls `ObjFree` once it has finished with the object. No other holder exists; no coordination is required.
- *Allocated for export*: the allocator passes a reference outward with appropriate capabilities and retains a reference with the `V` bit. When the time comes to revoke the export, `ObjRevoke` is called; recipients see `ESTALE` on subsequent dereferences and may recover at their discretion.

- *Held under contract by a service object*: the service is responsible for calling `ObjFree` on exported objects when its own contract ends.

The architecture's reliance on explicit revocation rather than tracing collection is deliberate. We considered and rejected a hardware- assisted reference-counting mechanism in early design, on the grounds that the cost of maintaining accurate counts across the crossbar would dominate any plausible savings; we expect higher-level languages on Object RISC to provide tracing collection in their runtimes when their semantics demand it.

4. Inter-Processor Communication Idioms

4.1 The Self Convention for Handlers

When firmware on a receiving processor dispatches a `SEND` handler, it constructs the entry context as follows:

- `O1` is set to a fresh reference to the recipient object itself, carrying the object's full `max_caps`. This is the *self* reference, by which the handler is expected to access its own state.
- `O2`–`O4` are set to the first three object references from the `SEND` payload as transmitted on the wire.
- `R4`–`R7` are set to the four integer payload words.

A SEND payload may carry up to four object references on the wire, as Volume IV specifies; the convention here delivers only the first three to the handler in object registers. A handler that requires the fourth invokes the firmware primitive `MessagePayloadOR4` (Volume VI Section 6, primitive number `0x205`), which retrieves the reference from the side-channel buffer where dispatch deposited it. We expect this case to be rare, and we expect a future revision to reorganize the dispatch convention to remove the asymmetry.

4.2 Asynchronous Notification

The simplest use of `SEND` is fire-and-forget: an event has occurred, the recipient should be told, no reply is expected.

```
; We hold a reference to the notification target in O9.
omov  o1, o9              ; recipient
onull o2                  ; no payload references
onull o3
onull o4
addiu r4, r0, EVENT_CODE  ; event identifier
addiu r5, r0, EVENT_DATA  ; event detail
send  o1                  ; fire
```

The handler at the recipient runs asynchronously, possibly on a different processor; the issuing task continues immediately. There is no acknowledgment; the handler may not have run by the time the issuing task observes any subsequent state.

4.3 Synchronous Reply

The pattern that most resembles a remote procedure call layers a reply object on top of `SEND`:

1. The caller allocates a small object to hold the reply, with capabilities `R|W|S|V`.
2. The caller attaches a receive queue to the reply object via `ReceiveQueueAttach`, so that incoming `SEND`s to the reply are delivered to a queue rather than to a handler.
3. The caller derives a send-only capability (`S` alone) on the reply object, suitable for passing to the callee.
4. The caller issues a `SEND` to the service, including the send-only reply capability in the payload.
5. The caller polls the reply queue via `ReceiveQueuePoll` with an appropriate timeout.
6. On reply, the caller reads the response payload from the registers loaded by `ReceiveQueue–Poll`, frees the reply object, and returns.

The full sequence is given in the worked example of Section 5.

5. A Worked Example: A Distributed Counter Service

5.1 The Service

A *Counter* is an object that holds a single 32-bit unsigned counter value and exposes three operations through `SEND`:

Opcode	Operation	Payload	Reply payload
`1`	`READ`	(none)	current count in `R4`
`2`	`INCREMENT`	(none)	new count in `R4`
`3`	`RESET`	new value in `R5`	old count in `R4`

Counters are allocated with type tag `0x4001` and the capability set `R|W|S|V|C` (`0x4B`). Clients receive references with `R|S` caps (`0x09`) — read for verification of locality, send for invoking operations.

The single shared handler routes on opcode and replies through the caller-supplied reply capability, which appears as `O2` of the handler's payload by the `SEND` convention of Section 4.1.

5.2 The Server

The server consists of an initialization routine, called once on the counter's home processor, and a handler, invoked once per incoming `SEND`.

5.2.1 Initialization

```
; counter_create(handler_code in O1) -> O1 = counter_ref, R2 = status
;
; Allocates a Counter object and installs the shared handler on it.
; The handler code object reference is supplied by the caller in O1.
```

```
counter_create:
    addiu sp, sp, -16
    sw    r31, 12(sp)
    omov  o9, o1                ; preserve handler code object

    ; Allocate the counter object: 4 bytes, type 0x4001, caps R|W|S|V|C
    addiu r4, r0, 4             ; length
    addiu r5, r0, 0x4001        ; type tag = Counter
    addiu r6, r0, 0x4B          ; caps = R|W|S|V|C
    call  #0x100                ; ObjAlloc
    bne   r2, r0, fail
    nop

    omov  o10, o1               ; preserve counter ref

    ; Initialize count to zero
    osw   r0, 0(o10)

    ; Install the handler: target = counter, handler code = O9, offset = 0
    omov  o1, o10               ; target
    omov  o2, o9                ; handler code object
    addu  r4, r0, r0            ; offset = 0 (entry of counter_handler)
    call  #0x200                ; InstallHandler
    bne   r2, r0, fail
    nop

    omov  o1, o10               ; return counter ref
    addu  r2, r0, r0            ; status = OK
    lw    r31, 12(sp)
    jr    r31
    addiu sp, sp, 16            ; (delay slot) deallocate frame

fail:
    onull o1
    lw    r31, 12(sp)
    jr    r31
    addiu sp, sp, 16
```

5.2.2 The Handler

```
; counter_handler:
;   Invoked by firmware when SEND arrives at a Counter object.
;   On entry:
```

```
;     O1 = self (counter ref, full caps)
;     O2 = reply object (send-only cap), or null if caller wants no reply
;     R4 = opcode (1 = READ, 2 = INCREMENT, 3 = RESET)
;     R5 = data (RESET only)
;   Runs as a freshly-spawned user-mode task. Returns by TaskExit.

counter_handler:
    addiu sp, sp, -16
    sw    r31, 12(sp)
    sw    r16, 8(sp)

    ; Branch on opcode
    addiu r16, r0, 1
    beq   r4, r16, do_read
    addiu r16, r0, 2            ; (delay slot) prepare next compare
    beq   r4, r16, do_increment
    addiu r16, r0, 3            ; (delay slot)
    beq   r4, r16, do_reset
    nop                         ; (delay slot)

    ; Unrecognized opcode: terminate without replying
    j     terminate
    addiu r4, r0, 1             ; (delay slot) exit code 1 (error)

do_read:
    olw   r4, 0(o1)             ; load current count into reply payload
    j     send_reply
    nop                         ; (load delay slot doubled as branch DS)

do_increment:
    olw   r16, 0(o1)            ; load current count
    nop                         ; load delay slot
    addiu r4, r16, 1            ; new count
    j     send_reply
    osw   r4, 0(o1)             ; (delay slot) store new count

do_reset:
    olw   r4, 0(o1)             ; load old count for the reply
    nop                         ; load delay slot
    j     send_reply
    osw   r5, 0(o1)             ; (delay slot) store new value

send_reply:
```

```
    ; If no reply object was supplied, skip the SEND.
    oisn  r16, o2
    bne   r16, r0, no_reply
    nop

    ; Issue reply: recipient = O2, payload R4 carries result.
    omov  o1, o2
    onull o2                    ; clear unused payload references
    onull o3
    onull o4
    send  o1                    ; fire reply

no_reply:
    addu  r4, r0, r0            ; exit code 0 (OK)

terminate:
    lw    r16, 8(sp)
    lw    r31, 12(sp)
    addiu sp, sp, 16
    call  #0x001                ; TaskExit (does not return)
    nop
```

5.3 The Client

```
; counter_get(counter in O1) -> R2 = count, R3 = status (0 on success)
;
; Issues a synchronous READ to the counter and returns the current value.

counter_get:
    addiu sp, sp, -16
    sw    r31, 12(sp)
    omov  o9, o1                ; preserve counter ref

    ; Step 1: allocate a 4-byte reply object with R|W|S|V caps
    addiu r4, r0, 4
    addiu r5, r0, 0x4002        ; type tag = Reply
    addiu r6, r0, 0x1B          ; caps = R|W|S|V (no derivation)
    call  #0x100                ; ObjAlloc
    bne   r2, r0, fail
    nop
    omov  o10, o1               ; preserve reply ref

    ; Step 2: attach a receive queue of depth one to the reply object
    omov  o1, o10
```

```
    addiu r4, r0, 1             ; depth = 1
    call  #0x203                ; ReceiveQueueAttach
    bne   r2, r0, fail_free
    nop

    ; Step 3: derive a send-only capability on the reply for the server
    omov  o1, o10
    addiu r4, r0, 0x08          ; mask = S only
    call  #0x103                ; ObjDerive
    bne   r2, r0, fail_free
    nop
    omov  o11, o1               ; preserve send-only reply cap

    ; Step 4: SEND the READ request.
    omov  o1, o9                ; recipient = counter
    omov  o2, o11               ; payload OR1 = reply send-cap
    onull o3
    onull o4
    addiu r4, r0, 1             ; opcode = READ
    send  o1

    ; Step 5: poll the reply queue, unbounded wait.
    omov  o1, o10               ; reply object (with V cap)
    addiu r4, r0, -1            ; timeout = 0xFFFFFFFF (unbounded)
    call  #0x204                ; ReceiveQueuePoll
    bne   r2, r0, fail_free
    nop

    ; ReceiveQueuePoll delivers integer payload in R3-R6 (per Volume VI
    ; Section 6 convention). Our reply put the count in R4 of the SEND
    ; payload, which arrives as R3 here.
    addu  r2, r3, r0            ; return value = count

    ; Step 6: free the reply object and return.
    omov  o1, o10
    call  #0x101                ; ObjFree
    addu  r3, r0, r0            ; status = OK
    lw    r31, 12(sp)
    jr    r31
    addiu sp, sp, 16

fail_free:
    omov  o1, o10
```

```
    call  #0x101                ; best-effort free
    nop
fail:
    addu  r2, r0, r0            ; return value = 0
    addiu r3, r0, 1             ; status = generic error
    lw    r31, 12(sp)
    jr    r31
    addiu sp, sp, 16
```

5.4 Trace of a Single READ Operation

Suppose the counter object lives on processor 3 and the calling task runs on processor 7. A trace of `counter_get` proceeds as follows:

1. *On processor 7.* The client allocates the reply object via `ObjAlloc`. Firmware on processor 7 selects a free slot in its local object table, allocates four bytes of local memory, and constructs a reference whose home is `7`.
2. *On processor 7.* `ReceiveQueueAttach` installs a per-object queue in firmware-managed memory and configures the local handler- dispatch to enqueue rather than spawn a task on receipt.
3. *On processor 7.* `ObjDerive` constructs a new reference to the reply object with capabilities reduced to `S` alone. The reference has the same generation, home (`7`), and index as the original.
4. *On processor 7.* The `SEND` instruction dispatches its payload to the crossbar interface. The payload's recipient is the counter reference, whose home field is `3`; the crossbar routes the `SEND_DELIVER` packet to port 3.
5. *On processor 3.* The crossbar interface receives the `SEND_DELIVER` packet and hands it to firmware. Firmware reads the recipient reference, looks up the descriptor in the local object table, finds the installed handler, spawns a task at the handler's entry point with `O1` set to a full-caps self reference and `O2` set to the reply send-cap from the payload, and sets `R4` to `1` (the opcode).
6. *On processor 3.* `counter_handler` runs. The `do_read` branch is taken; `OL W` fetches the count word, the descriptor cache hit is free, and the value is placed in `R4` for the reply.
7. *On processor 3.* The handler executes `SEND O1` to fire the reply. The recipient is the reply object on processor 7. The crossbar routes the packet back to port 7.
8. *On processor 7.* Firmware delivers the reply to the queue previously attached. `counter_get` is sleeping in `ReceiveQueuePoll`; firmware unblocks it.
9. *On processor 7.* `counter_get` reads the count from `R3`, frees the reply object via `ObjFree` (which increments the reply object's generation and returns the slot to the local pool), and returns to its caller.

Total architectural latency on the reference 16-port crossbar at 16 MHz is approximately 6 microseconds for the round trip, dominated by the two crossbar traversals at roughly 1.75 microseconds each plus the handler invocation at 1.4 microseconds.

6. Compiler Notes

6.1 Filling Delay Slots

The reference toolchain fills branch delay slots and load delay slots through a peephole pass that runs after instruction selection. The compiler considers, in order:

1. An instruction from the basic block following the branch that does not depend on the branch's outcome.
2. An instruction from the basic block targeted by the branch, when the branch is predicted taken and the duplicated instruction has no observable side effect on the not-taken path.
3. A `NOP`, when neither of the above applies.

The observed fill rates on the firmware reference workload are 60% for branch delay slots and 70% for load delay slots. We expect both numbers to improve as the toolchain matures.

6.2 Object Register Allocation

Object register allocation is performed separately from general register allocation, with its own live-range analysis. Because object registers cannot be spilled to memory, the allocator falls back, when register pressure exceeds the available class, to one of the strategies of Section 2.4: restructuring is not within its purview, so in practice it generates either an array-marshalling sequence (the preferred strategy) or, for compilations that link against a spill-supporting firmware, a sequence of `ORegSpill`/`ORegRestore` calls.

In our measurements on representative C code from the firmware reference workload, the array-marshalling fallback is engaged in fewer than 1% of compiled functions; the spill-call fallback is engaged in fewer than 0.1%.

6.3 Generating CALL Sequences

A `CALL` to a firmware primitive is emitted by the compiler as a single instruction; the calling sequence is otherwise standard. The compiler is permitted to assume that all caller-saved registers are preserved across `CALL`, by the convention of Volume VI Section 2.2. This permits register allocation across `CALL` boundaries that would not be permitted across an ordinary `JAL`, and is the principal performance advantage of `CALL` relative to a software-implemented trap.

The compiler emits a `CALL` to a primitive number that the linker resolves at link time against the firmware's exported primitive table. The architecture's reservation of the primitive number into the instruction itself (rather than into a register) is what makes this resolution possible without runtime indirection.

7. Performance Considerations

7.1 Local versus Remote Object Access

A local object access through `OL/OS` against a descriptor-cache hit costs the same as an ordinary load or store: one cycle in the common case. A remote access against a descriptor-cache hit is two crossbar traversals plus the round-trip through the home processor's memory hierarchy — approximately 28 cycles in the reference 16-port configuration at 16 MHz. The ratio of about 14 to one is the principal performance consideration when deciding whether to migrate an object or to map a copy of it locally.

A descriptor-cache miss adds approximately a further 30 cycles for the descriptor fetch round trip; subsequent accesses against the same descriptor are then at the cached cost. Programs that touch many distinct remote objects in close succession are accordingly far more expensive than programs that touch the same handful of remote objects repeatedly.

7.2 Descriptor Cache Locality

The descriptor cache is shared across all tasks resident on a processor. A task that begins execution on a processor where another task has recently used a related object inherits the warm cache; a task that migrates to a cold processor pays a sequence of misses before its working set is restored. For latency-sensitive task groups, the firmware primitive `TaskBindProcessor` (Volume VI Section 4) is the standard tool for asserting locality.

7.3 SEND Buffering and Back-Pressure

A `SEND` whose destination port is congested stalls until the crossbar accepts the packet. For applications that can tolerate the delay, this is harmless; for applications that require throughput under burst, the firmware primitive `SendBuffered` (Volume VI Section 6) absorbs the stall into a software queue, returning `EAGAIN` only when the queue itself is full.

The `SendBuffered` queue lives in the issuing task's address space and is sized at firmware allocation time. The reference firmware defaults to a queue of sixteen messages per task; tasks that require larger queues request them at task creation through an extension parameter not described in this volume.

Colophon

This reference comprises Volumes I through VII of the Object RISC architecture documentation, Revision 0.1, dated 1986. The volumes have been combined into a single document for ease of navigation and cross-reference. Page numbering is sequential across the work; the table of contents is hyperlinked.

— *The Object RISC Architecture Group*